

전역 조명

리얼타임 렌더링 4판 · 11장 Global Illumination

1차시 : 복습 ~ 일반적인 전역 조명 모델

2차시 : 주변 폐색 (Ambient Occlusion), 방향성 폐색 (Directional Occlusion)

3차시 : 확산 전역 조명, 반사 전역 조명

4차시 : 기타 & 최신 엔진 UseCase

오늘의 순서 - 1차시

- 00 서론 — 컴퓨터 그래픽스처럼 보이지 않기
- 01 복습, GI와는 무엇이 다른가 — 7·9·10장
- 02 렌더링 방정식과 전역 조명 알고리즘 — 11.1
- 03 일반적인 전역 조명 알고리즘 — 11.2

서론

“컴퓨터 그래픽스처럼 보이면, 좋은 컴퓨터 그래픽스가 아니다.”

— 제레미 번, 『디지털 라이팅 & 렌더링』

CG를 다루는 고전적인 교과서 포지션의 서적에서 인용한 문구라고 하고
프로그래밍지식보다는 그냥 사실적인 조명, 그림자, 텍스처, 라이팅, 렌더링 기법을 다루는 책이라고 합니다.

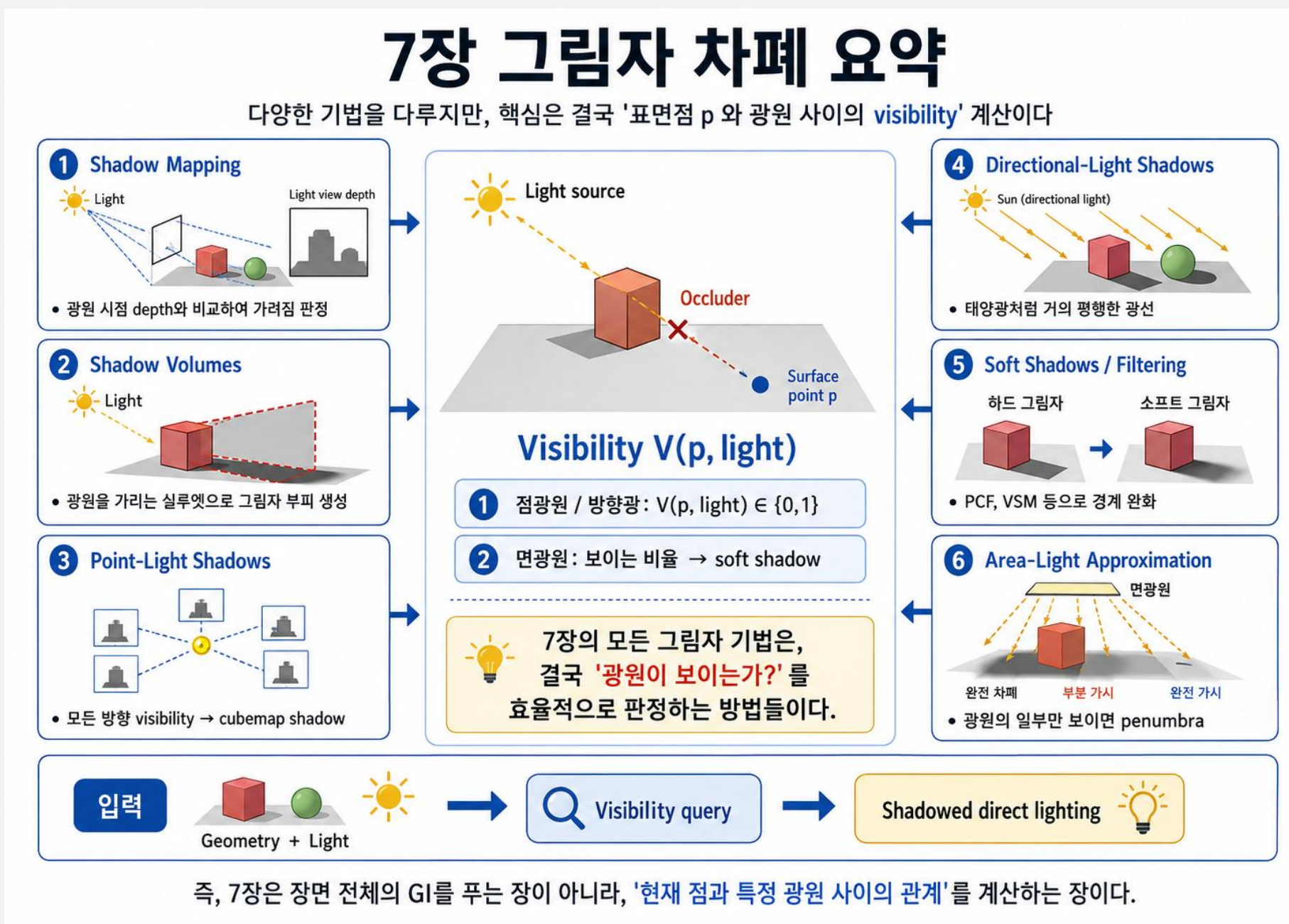
3D 아티스트를 위한 지식과 TA, 그리고 비주얼 프로그래머에게 도움이 되는 서적으로 보이는 내용들이 많이 있음
(물론 게임에서 사용하는 지식은 일부가 될 것 같습니다. 아 슬프구나)

복습 — 7·9·10

본격적인 GI파트를 다루기 전에
그림자, PBR, Local Illumination 파트에서 다룬 내용과의 연관점, 차이점을 먼저 확인하고
앞으로 보게 될 GI파트에서 해결하고자 하는 문제가 무엇인지 확인합시다.

7장 · 그림자

Shadow Map, Shadow Volume등으로 다른 오브젝트가 드리우는 그림자를 계산했지만, 이때 계산한 그림자는 직접광에 대한 가시성만을 판단하는 그림자였습니다.

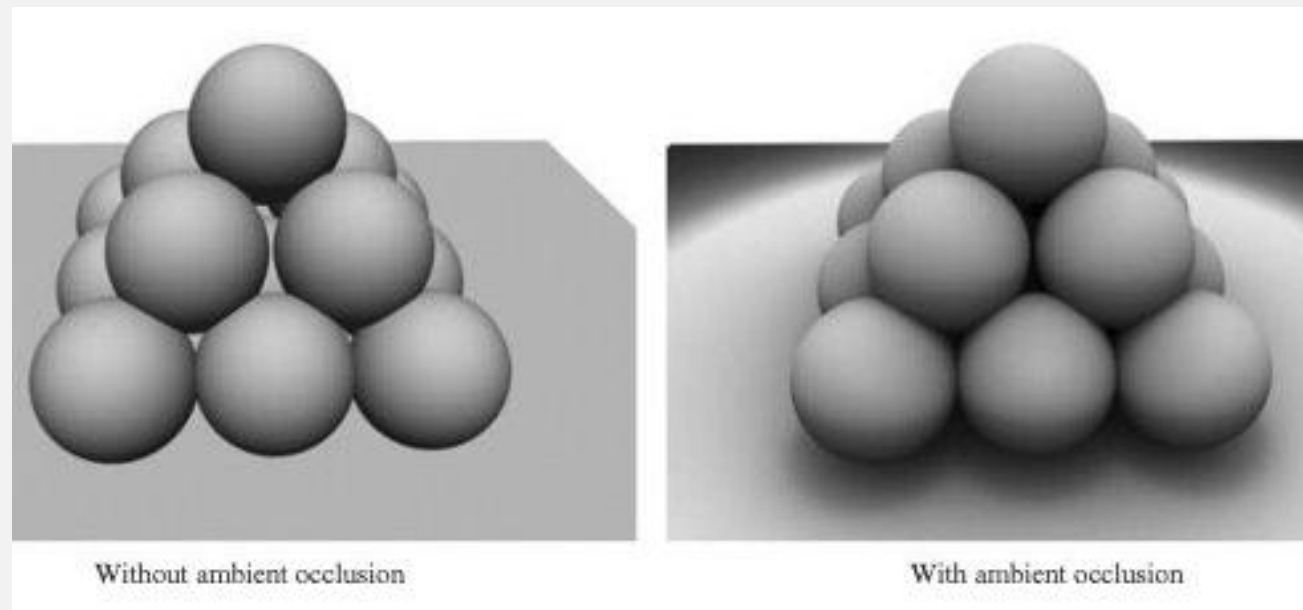


7장 · 현실의 빛과 어긋나는 지점

7장에서 계산한 것

지정한 광원 방향의 가시성

광원은 점(Point)거나 방향(Directional)이었고,
판정은 이진 값으로 처리되었습니다 (가려짐/안가려짐)
광원이 여러 개일 경우, 각 광원마다의 가시성을 판단했습니다.



광원의 위치 (위쪽) 기준으로 직접광의 차폐만을 고려했을 때와 간접광의 차폐를 비교한 모습

실제로 일어나는 일

다른 물체가 반사하는 빛도 차폐 될 수 있다

현실세계는 직접적인 광원 (Light Source)외에도, 대부분의 물체가 빛을 반사합니다.
그렇기 때문에, 동일하게 직접광을 받는 상황에도 직접광이 없는 방향의 차폐가
그림자를 만들게 됩니다.

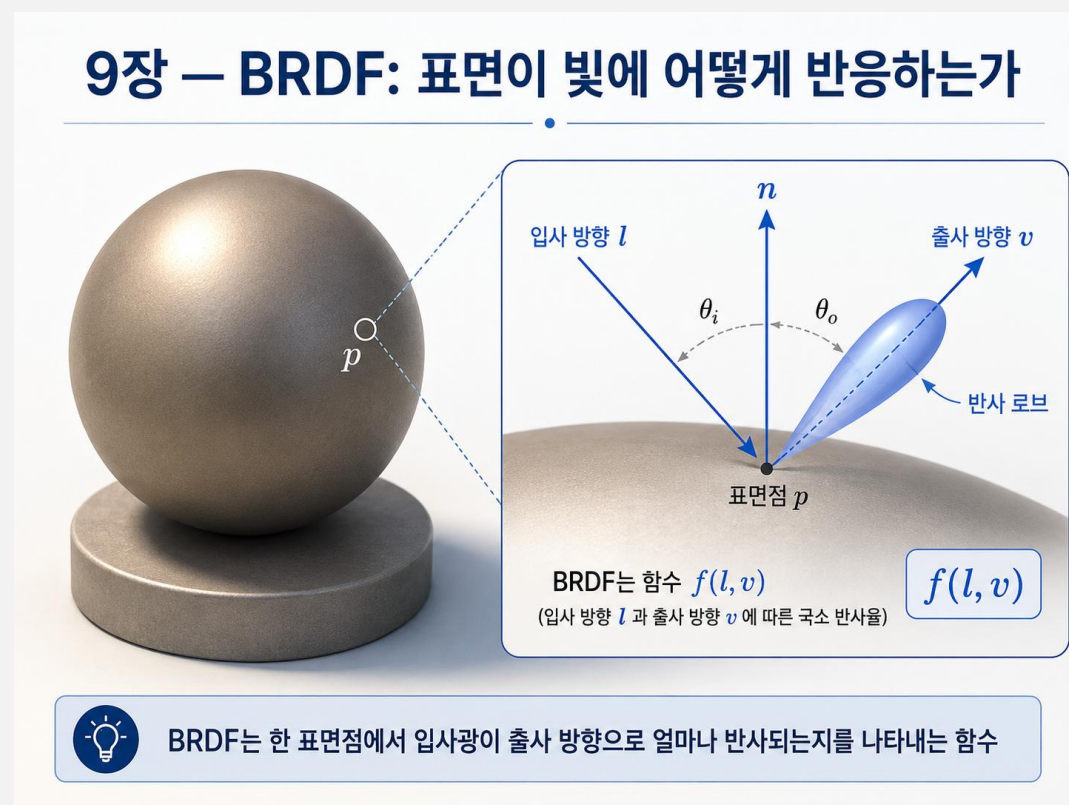
9장 · BRDF

양방향 반사도 분포 함수

$f(l, v)$

방향 l 에서 들어온 빛이 방향 v 로 얼마나 나가는가에 대한 함수
재질에 따라 특수한 함수를 사용하거나, 미세 표면을 고려하는 등의 내용이 있었지만
재질과 입사/출사 방향이 동일한 경우 언제나 같은 값을 냄.

입사 방향 l 에서 광량이 얼마나 들어오는지에 대해 다룬건 아니고, 단순히 재질(material)의 특성을 다룬 파트라고 이해



9장 · BRDF

반사율 방정식 · 9장

$$L_o(p, v) = \int_{l \in \Omega} f(l, v) L_i(p, l) (n \cdot l)_+ dl$$

9장에서 다룬 항은 반사율 방정식을 기준으로 위에서 푸른빛으로 강조한 항이 됩니다.
빛이 어디서 들어오는지에 대해서는 다루지 않았습니다.

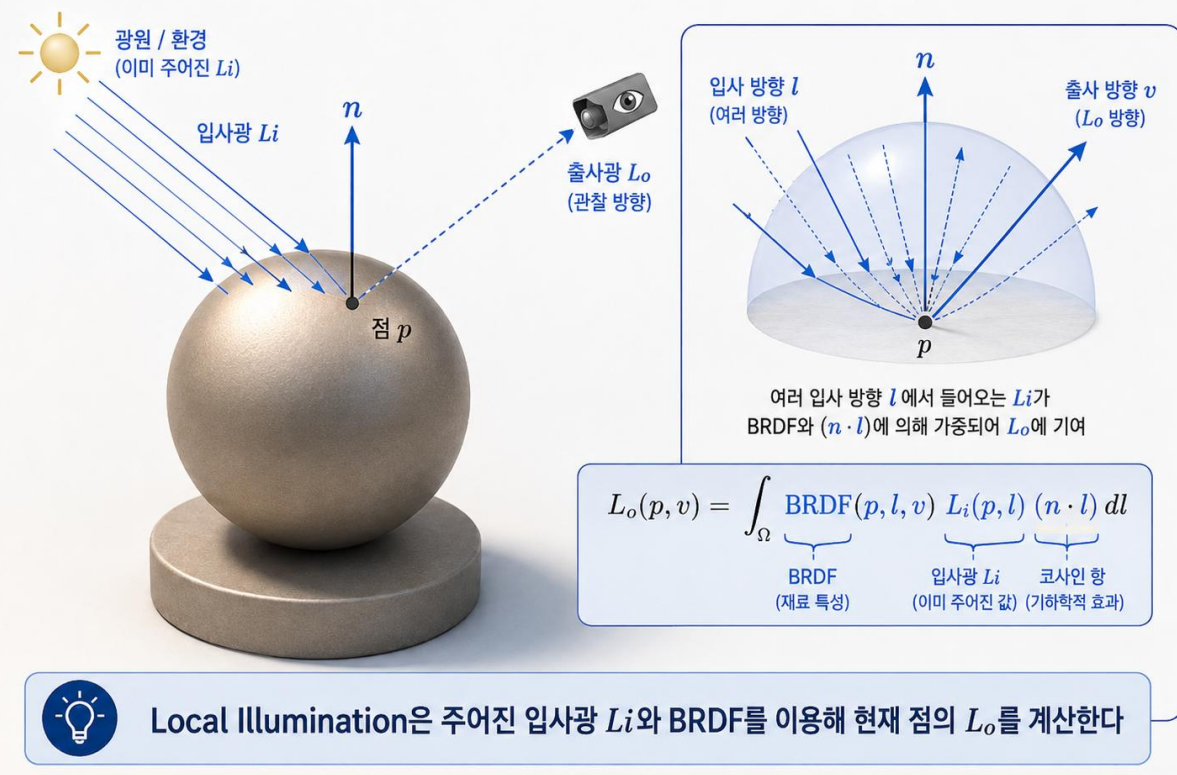
10장 · Local Illumination

10장에서 다룬 것은 조명 항

$$L_o(p, v) = \int_{l \in \Omega} f(l, v) L_i(p, l) (n \cdot l) dl$$

10장은 L_i 를 알고 있는 값 (프로그램기준으로는 알 수 있는 값)으로 놓고 반사율 방정식을 푸는 방법에 대해 다루었습니다.
점광원/방향광만을 고려하는 경우에는 적분은 유한한 합으로 나눌 수 있고
환경 조명인 경우에는 미리 적분해둔 텍스처 (또는 SH등)를 읽어 옵니다.
 L_i 가 유한하거나, 장면의 변화/상태와 무관하게 주어질 수 있다는 가정이 지역 조명의 핵심 구분 방법입니다.

10장 — Local Illumination: 주어진 L_i 와 BRDF를 적분



10장 · 환경 조명이 저장한 것

환경 맵

무한히 먼 곳에서 오는 입사 광도를 방향의 함수로 저장합니다.
위치는 인자가 아닙니다.

셋 다 “빛이 어디서 오는가”를 장면과 무관한 함수로 고정합니다.
물체가 움직여도, 옆에 빨간 벽을 세워도 이 함수는 변하지 않습니다.

방사 조도 맵

코사인 가중 적분을 미리 끝내 둔 확산 반사 전용 조명입니다.
실행 시에는 법선으로 한 번 조회합니다.

구면 조화

Image-Based-Light 대신 낮은 주파수 조명을 아홉 개 남짓한 계수로 압축합니다.
저장과 보관이 싸고 회전도 쉽습니다.

11장 · 다룰 내용

항목	지역 조명 모델의 가정	실제 장면
광원까지의 거리	무한히 멀다 — 방향만 남는다	유한하고, 위치마다 방향이 다르다
물체 간 차폐	지정한 광원 방향에만 그림자	모든 입사 방향에서 간접광이 차폐 될 수 있음
물체 간 상호 반사	빛은 물체 표면에서 한 번만 튕긴다	벽의 색이 바닥으로 번진다 (간접광)
조명	조명은 고정된 텍스처거나 punctual light	물체가 움직이면 조명도 함께 바뀐다

렌더링 방정식과 전역 조명

9, 10장에서 다루지 않은 렌더링 방정식과 전역 조명에서 필요한 광선 투사함수에 대해 다룬다는 내용 다듬어서 쓰기

반사율 방정식에서 렌더링 방정식으로

9·10장 — 반사율 방정식

$$L_o(p, v) = \int_{l \in \Omega} f(l, v) L_i(p, l) (n \cdot l)_+ dl$$

11장 — 렌더링 방정식

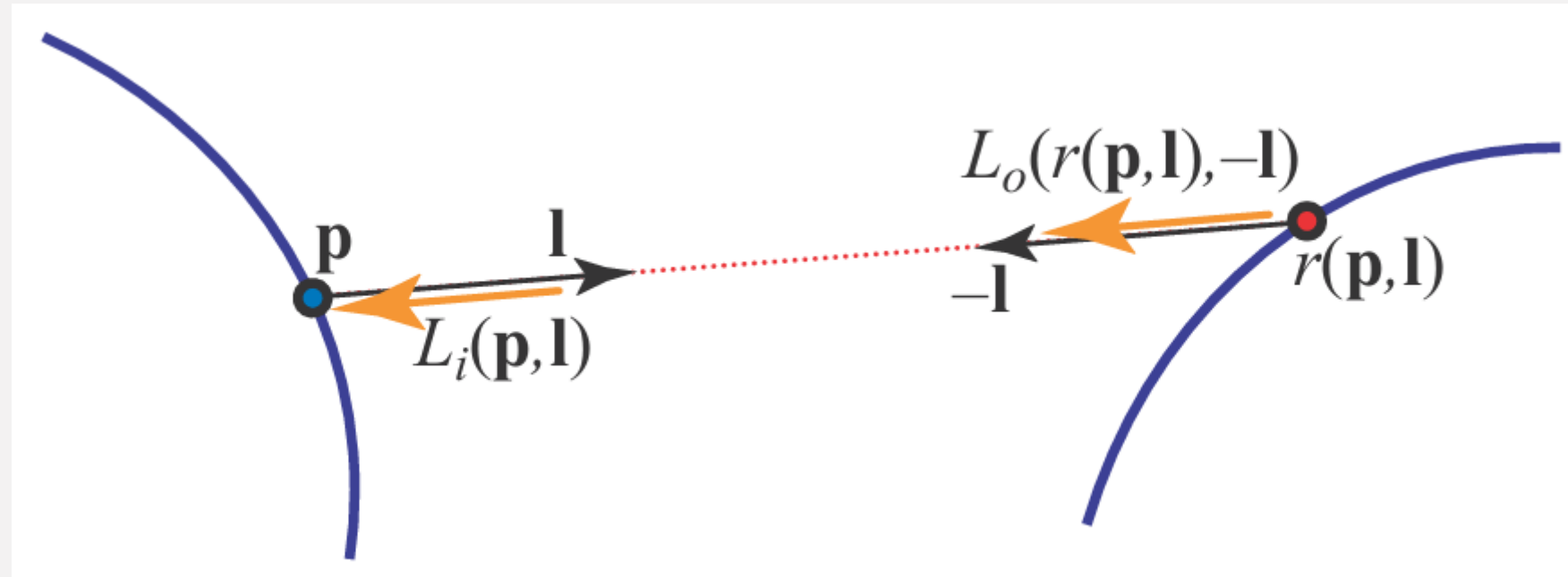
$$L_o(p, v) = L_e(p, v) + \int_{l \in \Omega} f(l, v) L_o(r(p, l), -l) (n \cdot l)_+ dl$$

9장과 10장에서 다룬 반사율 방정식은, 1986년 Kajiya가 제시한 렌더링 수식의 제한된 특수사례. 빛이 들어와서 반사한다고 했지만, 현실의 물체는 스스로 빛을 내는 경우도 있으며 (빨간 항) 물체에 들어오는 빛은 어디 허공에서 생성되는게 아니라 어딘가에서 방출되어 옵니다. (파란항)



빨간 항의 예시 : 스스로 빛을 내는 야광공룡 이건 못참지

광선 투사 함수 $r(p, l)$



$r(p, l)$ 은 점 p 에서 방향 l 로 광선을 쏘았을때 가장 먼저 만나는 표면 위의 점을 돌려줍니다.
기존의 반사방정식에서 다른 빛이, 어느 지점에서 얼마나 오는지 알기 위해서는 $r(p, l)$ 지점을 찾아야겠죠?

$r(p, l)$ 지점에서 p 방향으로 오는 빛(L_i)은 $L_o(r(p, l), -l)$ 이 됩니다.

r인 이유 : ray casting function입니다

- 반투명한 물체에 대해서는 다루지 않습니다. BRDF함수부터가 빛의 반사를 가정한 함수지 빛의 투과를 고려하지 않았습니다.
- 그렇기 때문에 ray cast하는 방향에서 오는 빛은 언제나 단일 지점임을 보장할 수 있습니다.
- 반투명한 물체는 14장에서 다룰 예정이니 잠시동안은 불투명한 세상을 상상합시다.

렌더링 방정식이 의미하는것

$$L_o(p, v) = L_e(p, v) + \int_{l \in \Omega} f(l, v) L_o(r(p, l), -l) (n \cdot l)_+ dl$$

$L_e(p, v)$ 는 이미 10장에서 광원의 방출을 다루는 방법을 보았으므로 생략합니다.
그런데 $L_o(p, v)$ 값을 구하려면 반구 전체 방향으로 ray cast를 시도해서 $L_o(r(p, l), -l)$ 를 구해야합니다.

그렇다면 각 L_o 값들을 구하기 위해서 다시 해당 지점들에서 반구 전체 방향으로 ray cast를 시도하고...
모든 빛이 흡수될때까지 재귀적으로 계산해야 합니다

저 L_o 계산하는게 해석적으로 풀리진 않을 것 같은데 아마 없겠죠? AI 피셜로는 제2종 프레드홀름 적분 방정식이라는데 모르겠음

렌더링 방정식의 성질

모든 광원의 emission이 2배가 되면, 음영 처리 결과가 2배 밝아지고
각 Light Source에 대한 재료의 반응은 다른 Light Source와 독립적입니다.

Light는 기본적으로 Additive하게 작동하기 때문에 (가산혼합)
광원별 결과를 계산 한 뒤 합산할 수 있음을 보일 수 있습니다.

ps 하셨던 분들은 기대값의 선형성 태그라던가, 그런곳에서 본 선형성이라는 단어가 와닿을듯

렌더링 방정식의 두 가지 핵심 성질

1) 방출 조명에 대해 선형(linear) 2) 각 광원의 재료 반응은 서로 독립적

1 선형성: 광원이 2배 밝아지면 결과도 2배

경로 기여도 2배

$$T(2L_A) = 2T(L_A)$$

직접광, 1-bounce, 2-bounce ... $LE \times 2$
모든 경로의 기여가 동일한 비율로 증가 $LDE \times 2$
 $LDDE \times 2$

2 광원들의 독립성: 각 광원의 기여는 따로 계산해서 더할 수 있다

A의 경로 (A의 기여) B의 경로 (B의 기여)

$$T(L_A + L_B) = T(L_A) + T(L_B)$$

A의 경로 기여 + B의 경로 기여 → 최종 Radiance

Light A의 존재가 Light B의 재료 반응을 바꾸지 않는다

3 직관적으로 보면

- 렌더링 방정식은 방출 조명 L_e 에 대해 선형이다.
- 광원별 light path의 기여는 서로 독립적이다.
- 따라서 광원별 결과를 계산한 뒤 합산할 수 있다.

$$L = (I - K)^{-1}L_e$$

장면의 geometry와 material이 고정되면, transport operator는 그대로이고 입력 조명만 선형적으로 반영된다.

L : 출사 복사도 (해)
 L_e : 자발광(방출) 복사도
 K : 광선 전송(재귀 반사) 연산자
 I : 단위 연산자

핵심: 모든 light path의 기여를 더한 것이 최종 밝기이며, 광원을 k 배 하면 각 path의 기여도 k 배가 된다.

실시간 렌더링에서 주로 지역 조명만 사용하는 이유

렌더링 방정식을 프레임마다 전부 풀 수는 없다

목표로 하는 픽셀 하나를 렌더링하기 위해 씬의 모든 지점에 대해 BRDF함수를 평가하고 적분하는 것은 실시간 내에 처리하기 어렵습니다. 정밀도를 낮추기 위해 반구를 샘플링 하는 비율을 낮추고, 낮은 광량을 가진 Light Pass계산을 생략하더라도 런타임 (60프레임 기준 16ms)환경에서 작동하기에는 현실적인 제약이 아닙니다.

GI는 전통적인 단일 **Rasterization Pass**의 **Fragment Shader**만으로는 완전한 **GI**를 계산하기 어렵다.

Rasterization 파이프라인의 Fragment Shader는 현재 rasterize된 surface point를 독립적으로 shading하는 데 최적화되어 있습니다. 반면 Global Illumination은 현재 점의 조명을 계산하기 위해 장면의 다른 surface point들의 조명·가시성 정보가 필요합니다. 이러한 지점들은 현재 화면에 fragment를 생성하지 않거나, 같은 pass에서 아직 shading 결과가 존재하지 않을 수 있으므로, 단일 Fragment Shader pass만으로 GI의 surface 간 light transport를 해결하기 어렵습니다.

(지역 조명은 Li가 주어진 값이기 때문에, 대부분의 데이터를 fragment 셰이더에서 알 수 있습니다)

- 그림자 처리도 그렇지만, 위와 같은 이유로 런타임 GI는 기본적으로 DrawCall이 내려지기전, 미리 계산하는 과정을 거치게 됩니다.
- 그게 이미 구성된 씬에서 미리 구워두는 방법이든, 런타임에 Shadow Map처럼 다른 정보를 굽게 되든...

Heckbert의 광자 경로 표기법

기호

L — 광원(light), 경로의 출발점

E — 눈(eye), 경로의 도착점

D — 확산 반사(diffuse)

S — 정반사(specular)

연산자

k^* — 0회 이상 반복

k_+ — 1회 이상 반복

$k?$ — 0회 또는 1회

$(k|k')$ — 둘 중 하나

는 UseCase들을 보고 기호에 매핑했을때 이해가 더 빠를 것 같기 때문에 지금은 넘깁니다.



전역 조명을 실시간에 넣는 두 가지 전략

전략 1

단순화 — 식의 일부에 강한 가정을 건다

조명이 모든 방향에서 균일하다고 보거나, 반사가 확산뿐이라고 보거나, 빛이 멀리 가지 않는다고 보는등의 제약으로 식을 단순화 합니다.

전략 2

사전 계산 — 변하지 않는 것을 미리 푼다

정적인 기하 구조와 정적인 광원 사이의 빛 전달을 미리 풀어 라이트 맵이나 계수로 저장하고, 실행 시에는 조회와 보간만 합니다.

어떻게 하드웨어에 호환되는 기법으로 단순하게 만들거나 미리 구워두지 (또는 둘다 해서 통치기) 같은 고민이 요즘 GI기술의 집합체 11.2장에서는 그나마 덜 다루고, 11.3장부터 적극적으로 위 두 주제를 다룰 예정입니다.

최근 GI 기술에는 최근에 풀었던 값을 캐싱해서 사용하는 방법에 대해서도 다루고, 별도의 하드웨어 유닛의 도움을 받아 병목을 해결하기도 합니다.

일반적인 전역 조명 알고리즘

10장에서는 반사율 수식을 푸는 다양한 방법에 중점을 두고, 들어오는 광도 L_i 의 분포를 가정하며 그것이 음영에 어떻게 영향을 미치는지 분석했었다. 이번 장에서는 방금 설명한 렌더링 수식을 풀도록 설계된 알고리즘 2개를 제시한다.

한 지점에 도달하는 광도는 다른 지점에서 방출되거나, 반사된 광도가 된다.

퀄리티가 굉장하지만, 기존에 설명했듯 런타임에 계산하기에는 무리가 있다.

그럼에도 해당 과정을 다루는 이유는

1. 정적이거나 / 부분적으로 정적인 씬에서 이런 알고리즘으로 사전처리해 나중에 사용할 수 있도록 저장할 수 있기 때문
2. 실시간 솔루션을 설계할때도 올바른 방법이 무엇인지 비교하거나, 유사한 유형의 추론을 적용할 수 있기 때문

특히 유사한 유형의 추론을 적용한다는 부분에 집중해서 일반적인 전역 조명 알고리즘을 보면 되겠다.

해당 파트에서 나오는 개념과, 이를 구현하기 위해 사용하는 아이디어들은 이후 각기 다른 GI 구현체들의 기본 개념이 될 수 있다.

렌더링 수식을 푸는 두가지 일반적인 방법은 유한 요소 해석(방법) 과 몬테 카를로 방법이 있다.

라디오시티(Radiosity)는 유한 요소 해석을, 다양한 레이트레이싱(RayTracing)은 몬테 카를로 방법을 기반에 두고 있다.

- 유한 요소 해석 : 공간을 작은 요소들로 분할하고, 각 요소별 물리 법칙을 식으로 세워 연립방정식을 푸는 형태. 고체, 유체, 열역학 등에서 사용한다고 함
- 몬테 카를로 방법 : 유명한걸로는 원의 면적을 계산하는것이 있음. RayTracing을 몬테 카를로 방법을 사용하면 임의의 방향으로 Ray를 쏘는것을 상상하면 된다

11.2 General Global Illumination

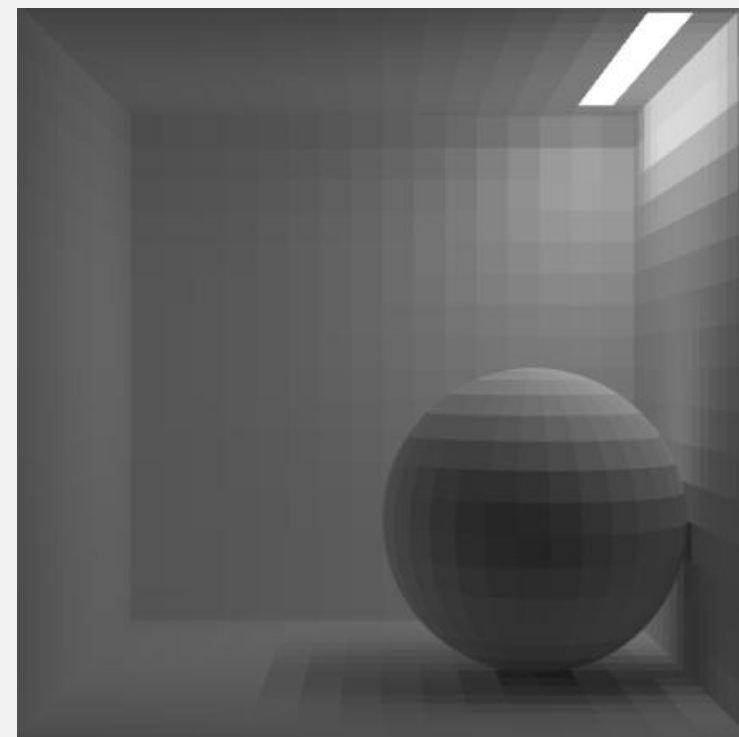
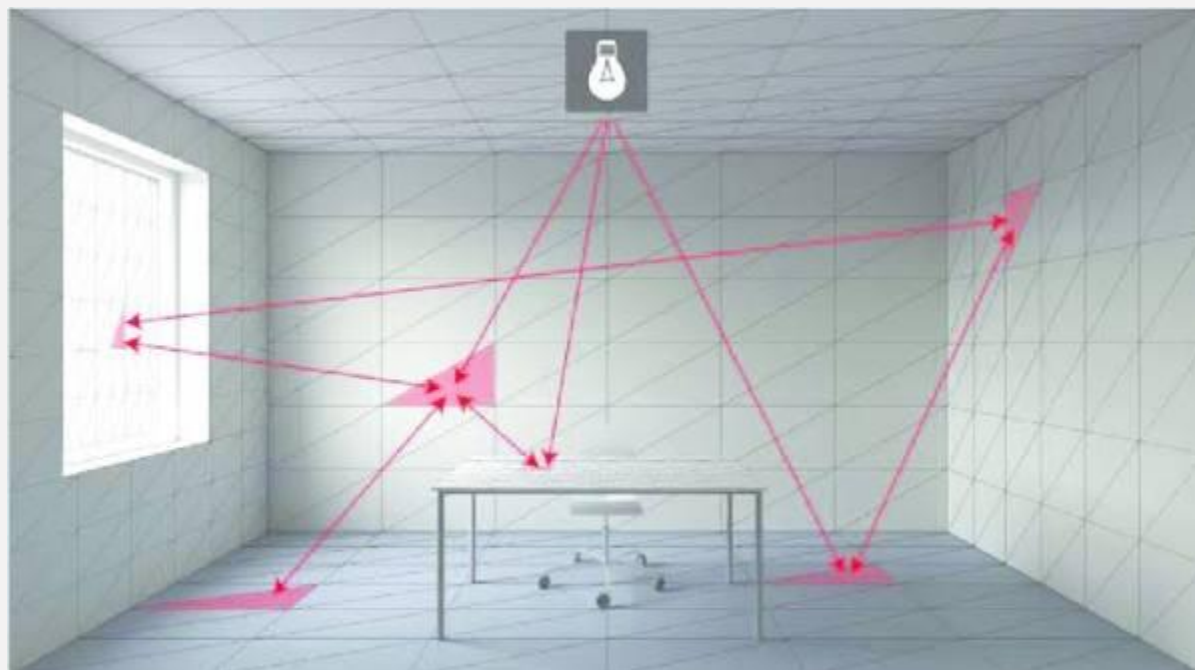
Radiosity

모든 표면이 **확산 반사**만 한다고 가정하면, 한 점의 밝기는 보는 방향과 무관해집니다. (Lambertian)

방향 의존성이 사라지면 미지수는 방향의 함수가 아니라 **위치만의 함수**가 됩니다.

장면을 패치로 나누고 패치들 사이의 에너지 교환을 연립방정식으로 풉니다.

열전달 공학에서 넘어온 접근으로, 1984년 코넬대에서 그래픽스에 처음 적용되었습니다.



Radiosity

패치 i 의 방사도

$$B_i = B_{ei} + \rho_i \sum_j F_{ij} B_j$$

F_{ij} 는 패치 i 를 떠난 에너지 중 패치 j 에 도달하는 비율입니다. 재질과 무관한 순수한 기하 문제입니다.

형태 계수 계산이 전체 비용을 지배하고, 패치 수의 제곱에 비례해 늘어납니다.

점진적 세밀화는 가장 밝은 패치부터 순서대로 에너지를 뿌려, 중간 결과를 미리 보여 줍니다.

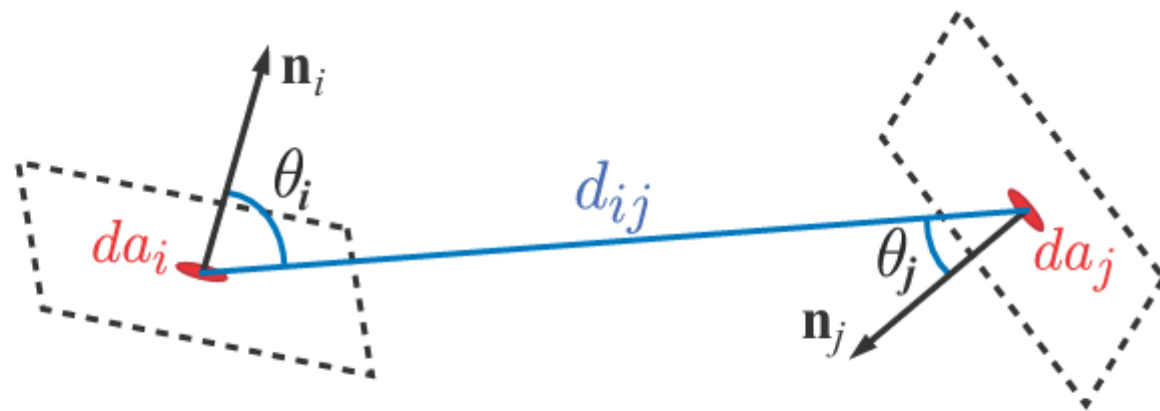
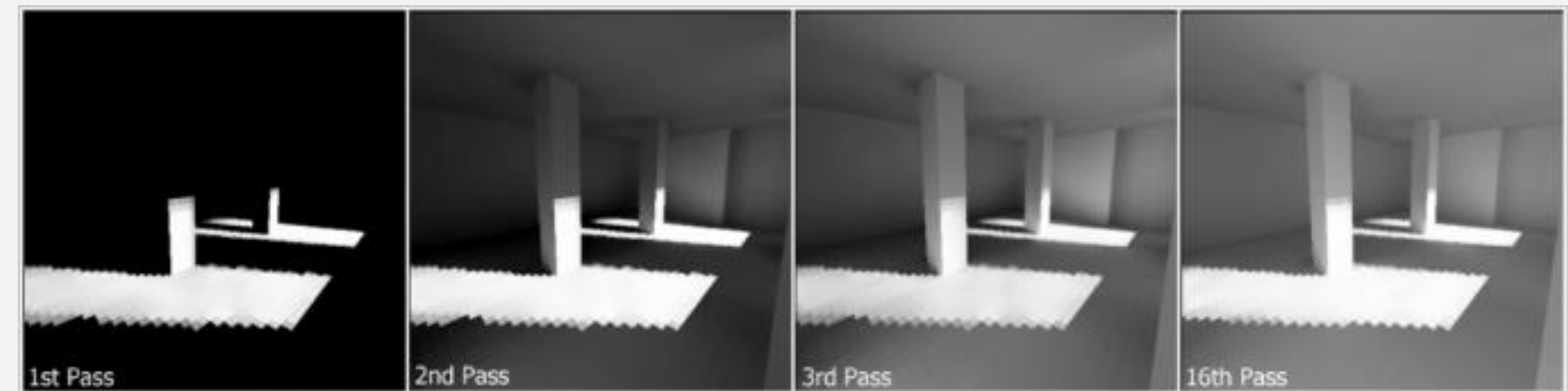


Figure 11.4. The form factor between two surface points.



As the algorithm iterates, light can be seen to flow into the scene, as multiple bounces are computed. Individual patches are visible as squares on the walls and floor.

Radiosity의 장단점

얻은 것

시점과 무관한 해

한 번 풀어 두면 카메라를 어디에 두어도 그대로 쓸 수 있음.
결과가 정점이나 텍셀에 저장되므로 실행 시 비용이 거의 없음.

잃은 것

정반사와 메시 독립성

방향정보를 손실하고 표면 patch가
받은 빛이 균일하게 반사된다 가정하였기 때문에
거울과 하이라이트를 표현하지 못함 (Diffuse 경로에만 특화)
메시 분할 정도가 품질이라 그림자 경계에서 계단이 드러남.

Ray Tracing

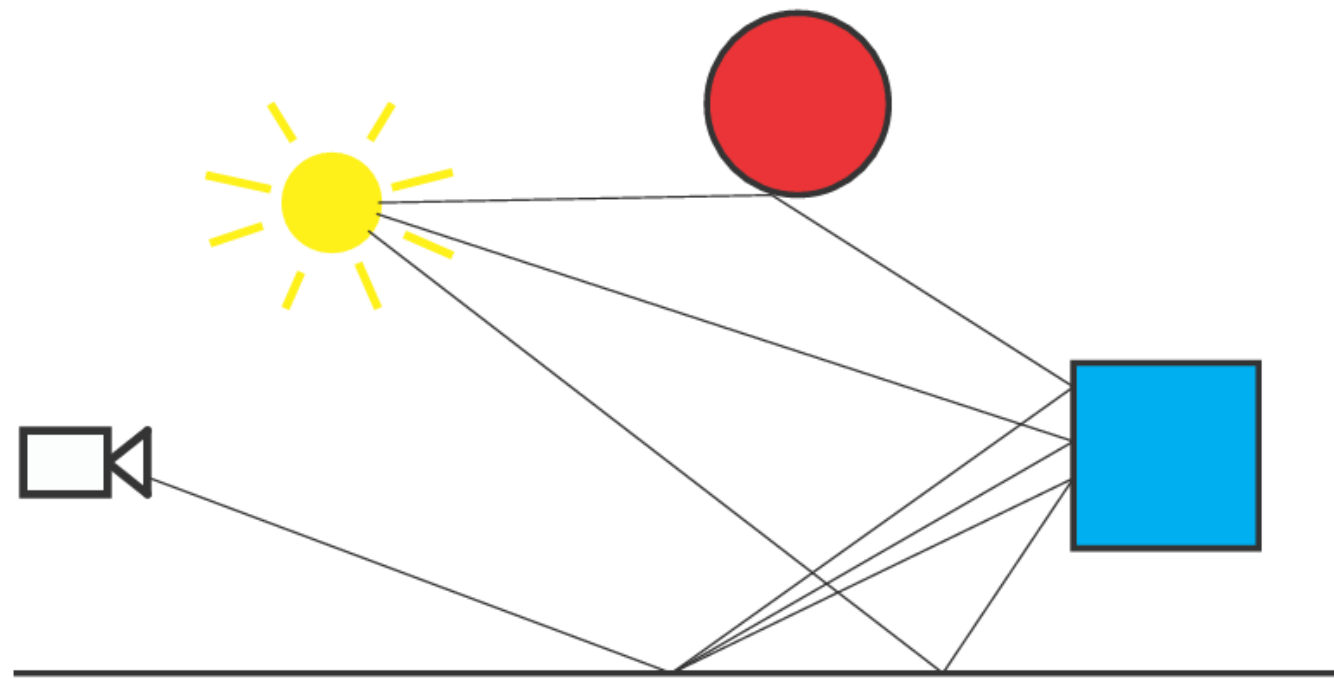


Figure 11.5. Example paths generated by a path tracing algorithm. All three paths pass through the same pixel in the film plane and are used to estimate its brightness. The floor at the bottom of the figure is highly glossy and reflects the rays within a small solid angle. The blue box and red sphere are diffuse and so scatter the rays uniformly around the normal at the point of intersection.

Ray Tracing의 장단점

얻은 것

재질과 상관없이 빛의 반사를 시뮬레이션 할 수 있음

RadioSity는 재질이 가진 glossy함이나, Mirror같은 반사 로브의 특징을 살릴 수 없었으나, Ray Tracing은 BRDF의 형태에 크게 구애받지 않고 사용할 수 있는 기술이 된다.

엄청난 샘플링 비용을 줄이기 위해
대부분의 빛이 방향이 들어오는 방향으로 더 많은 광선을 쏘거나
BRDF 함수의 특성에 따라 샘플링 전략을 변경하는 등의 기술들이 연구되었고, 연구되는 중
이후 GI파트에서도 약간 다루겠지만, 적은량의 샘플링을 어떻게 커버할지도 중요한 주제

잃은 것

샘플링 비용과 노이즈

모든 방향의 Ray를 Trace할 수 없고, trace의 수가 줄어들수록
노이즈의 분산이 커지게 된다.

동일한 방법으로 샘플링한다고 가정했을때 노이즈(표준편차)를
2배 줄이기 위해서는, 4배 더 많은 샘플이 필요하게 된다.

Radiosity의 장점으로 쓴 시점과 무관한 해는 RayTracing의 단점으로 쓰진 않았습니다.
Unity / Unreal에서 static한 오브젝트에 대해 미리 굽는 과정에서는 RayTracing을 사용하고
있고 (거울등의 처리때문에라도), 이때는 광원 (Light Sources)에서 Ray를 쏘아 장면
에 반영되는 광량을 계산해, 물체의 표면에 반영합니다.

1차시 마무리

주변 폐색 하다가 어디서 끊어야할지도 모르겠고 준비도 미흡해서 일단 여기서 끊기
앞 파트 발표 시간 얼마나 소요되는지 보고 판단하기

오늘의 순서 - 2차시

- 00 서론 — 지역 조명은 어디서 티날까?
- 01 주변 폐색 — 11.3
- 02 주변 폐색 — 11.3 * 오타 아닙니다. 주변 폐색만 하루종일 합니다
- 03 방향성 폐색 — 11.4

서론

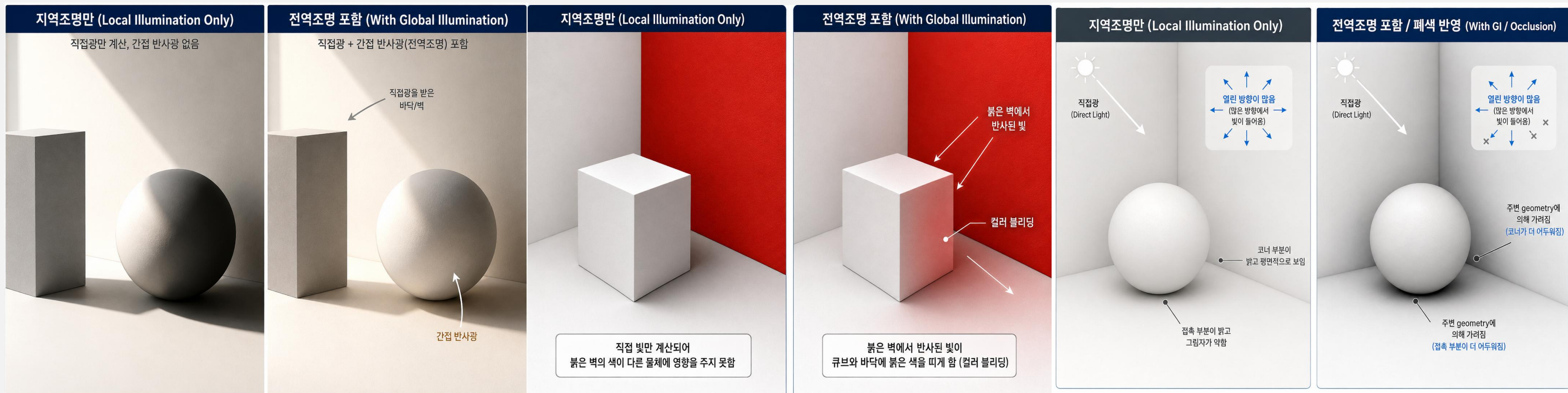
11장 1차시에 다룬 일반적인 전역 조명 알고리즘은 무겁지만,
7장에서 다룬 그림자와 10장에서 다룬 지역조명만으로는 물체 표면에서 반사된 간접광을 표현하지 못합니다.

아래 예시는 간접광이 만드는 대표적인 시각적 효과 3가지를 설명합니다. * 이미지는 AI 생성이라 실제 렌더링 결과와 차이가 있을 수 있음

Indirect Fill : 직접광이 닿지 않더라도, 간접광을 닿아 밝아질 수 있음

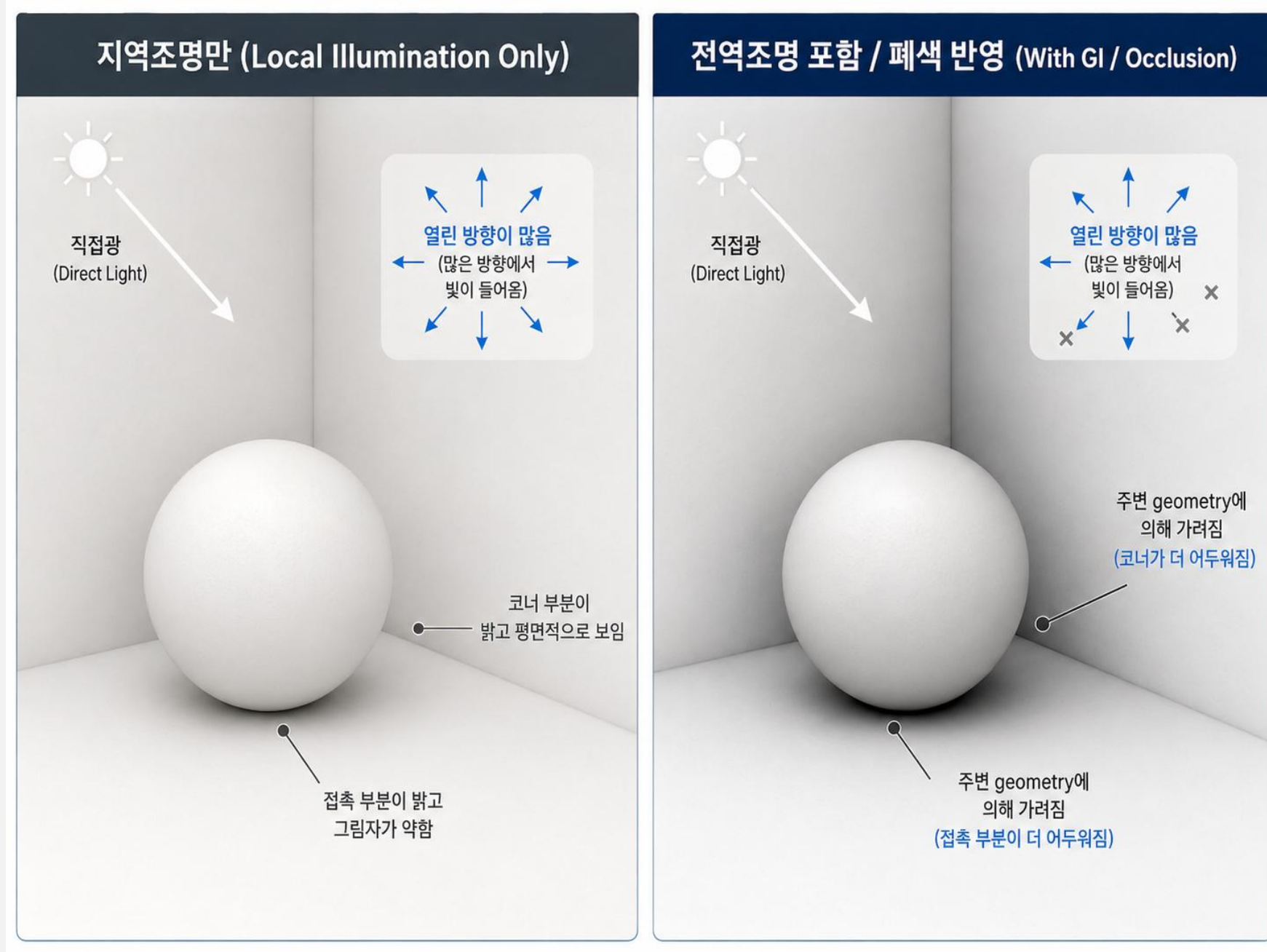
Color Bleeding : 표면의 색상이 다른 표면의 반사광의 영향을 받음

Occlusion : 빛이 들어오는 영역이 좁아지는 경우 상대적으로 어두워짐



서론

11.3 ~ 11.4는 주변광의 차폐를 어떻게 처리할 것인지에 대한 맥락을 담고 있습니다.



주변 폐색 (Ambient Occlusion)

1차시에 다룬 일반적인 전역 조명 알고리즘은 런타임에 사용하기에는 계산 비용이 너무 크고 복잡합니다.

이번 장에서 다룰 주변 폐색 (Ambient Occlusion)은 기본적으로면서도 대표적인 GI 효과입니다.
영화 진주만 (2001)에서 비행기에 사용하기 위한 환경 조명의 품질을 개선하기 위해 개발되었다고 합니다.

매번 1900년대, 심하면 1800년대 사람들 따라가느라 자괴감 느껴지고 벅찼는데, GI는 실사용 영역에서는 2000년대 기술들 나와서 좀 감동

이때도 많은 영화의 렌더링에는 RayTracing이 사용되고 있었지만, 진주만은 (그 당시 기준으로) 씬이 복잡해서 메모리 문제를 해결하기 위한 다른 수단이 필요했습니다.

- * RayTracing 기반 렌더링을 위해서는 씬 전체가 메모리에 올라가야만 했습니다. 반사광은 반사되는 위치의 재질 정보에 따라서도 달라지기 때문
- * 하지만 단순 차폐 검사는 오브젝트 단위로 미리 구워둘 수 있고, 재질의 변경에도 영향을 받지 않습니다. 왜 그런지는 이제 알아보도록 합시다.



진주만 영화에서 사용된 AO 효과의 예시 ([출처](#))

11.3 Ambient Occlusion

주변 폐색 이론 (Ambient Occlusion Theory)

주변 폐색 이론의 이론적 배경은 반사율 수식에서 유도할 수 있습니다.
단순화를 위해 Lambertian 표면을 가정합니다.

이러한 표면에서 나가는 radiance L_o 는 surface irradiance E 에 비례합니다. *irradiance = 복사조도, 표면이 받은 빛 (에너지)의 양

반사율 방정식

$$L_o(p, v) = \int_{l \in \Omega} f(l, v) L_i(p, l) (n \cdot l)^+ dl$$

Lambertian BRDF

$$f(l, v) = \frac{\rho_{ss}}{\pi}$$

Surface irradiance

$$E(p, n) = \int_{\Omega} L_i(p, l) (n \cdot l)^+ dl$$

$$L_o(p, v) = \frac{\rho_{ss}}{\pi} E(p, n)$$

* 10장에서 다룬 램버시안

여기에 더해, 단순화를 위해 들어오는 모든 방향 l 에 대해, incoming radiance가 일정하다고도 가정해봅시다. $L_i(p, l) = L_A$

*위 가정을 diffuse illumination(확산 조명)만 존재하는 환경이라고도 할 수 있습니다.

$$E(p, n) = \int_{\Omega} L_A (n \cdot l)^+ dl$$

한글본(4판)에서는 다시 말하지만 단순함을 위해 어찌구로 써놨는데
Lambertian 표면과 incoming radiance가 일정하다는건 서로 다른 가정 두개입니다.

$$E(p, n) = L_A \int_{\Omega} (n \cdot l)^+ dl = \pi L_A$$

* 수식 (11.6)

일정하고 균일한 조명이 Lambertian 표면에 들어오면 irradiance (결과적으로는 radiance도) 표면 위치와 법선(normal)에 의존하지 않으며, 오브젝트 전체에 일정합니다.
이는 물체의 형태에 따른 명암이 보이지 않게 되며 flat하게 보이게 됩니다.

주변 폐색 이론 (Ambient Occlusion Theory)

방금 유도한 식은 가시성을 고려하고 있지 않습니다.

반구의 모든 방향에서 동일한 빛이 들어오더라도, 오브젝트 본인에게 가리거나 다른 오브젝트에 의해 빛이 차단될 수 있습니다.

이런 방향에서는 L_A 가 아닌 다른 radiance가 들어오게 됩니다.

여기서 또 **단순화를 위해 가려진 방향에 대한 radiance는 0이라고 가정**합니다.

차단된 오브젝트에서 오는 반사광 / 해당 방향에서 오는 산란된 빛과 같은 요소를 무시하지만 결과를 단순화 할 수 있습니다.

이는 Cook과 Torrance가 처음 제안한 다음의 식을 얻게 됩니다.

$$E(p, n) = L_A \int_{\Omega} v(p, l)(n \cdot l)^+ dl \quad * \text{수식 (11.7)}$$

여기서 $v(p, l)$ 은 p 에서 l 방향으로 투사되는 광선이 차단되면 0이고 차단되지 않으면 1인 가시성 함수(visibility function)입니다.

이 가시성 함수의 정규화된 코사인 가중치 적분(normalized, cosine-weighted integral)을 이번 장에서 다룰 주변 폐색(AO)이라고 합니다.

$$k_A(p) = \frac{1}{\pi} \int_{\Omega} v(p, l)(n \cdot l)^+ dl \quad * \text{수식 (11.8), 주변 폐색}$$

완전히 가려진 표면의 경우 0, 어떤 방향도 가려지지 않은 경우 1이 됩니다.

$n \cdot l = \cos\theta$ 이기 때문에, 빛이 들어오는 방향에 따라 가중치가 다르게 됩니다 (코사인 가중치)

주변 폐색 이론 (Ambient Occlusion Theory)

주변 폐색 k_A 을 정의했기 때문에, 폐색을 고려한 irradiance를 구할 수 있게 됩니다.

$$E(p, n) = L_A \int_{\Omega} v(p, l)(n \cdot l)^+ dl \quad k_A(p) = \frac{1}{\pi} \int_{\Omega} v(p, l)(n \cdot l)^+ dl$$

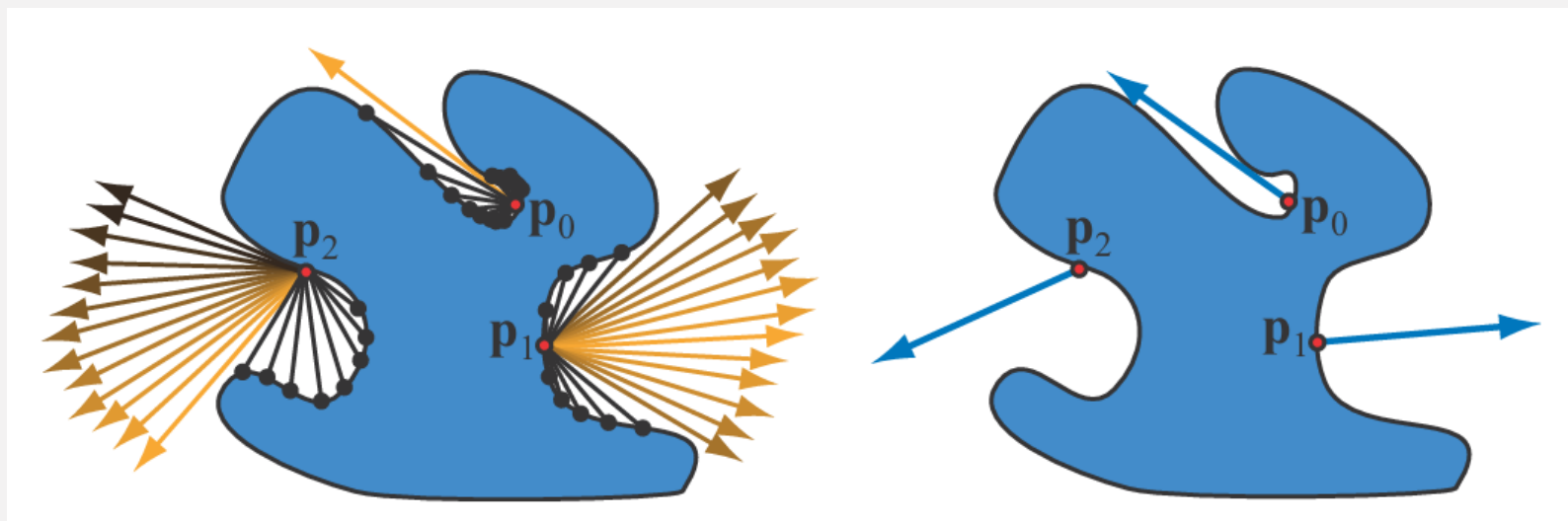
$$E(p, n) = k_A(p)\pi L_A \quad * \text{수식 (11.9)}$$

이제 표면 위치 p 가 바뀌면 주변 폐색 k_A 이 변경되기 때문에, irradiance가 위치에 따라 변경되게 됩니다.
이는 그림의 예시에서 볼 수 있듯이 훨씬 현실적인 결과로 이어집니다.

오목한 표면 위치는 k_A 값이 낮기 때문에 어두워지게 됩니다. (그림의 p_0 과 p_1 비교)

가시성 함수 $v(p, l)$ 이 통합될때 코사인($n \cdot l$)값에 따라 가중되어 표면 방향도 영향을 미칩니다.
 p_1 과 p_2 는 동일한 폐색되지 않은 각도를 가졌음에도 p_1 이 더 높은 값을 가지게 됩니다.

이후 다룰 내용부터는 간결한 표현을 위해 표면위치 p 에 대한 의존성은 명시적으로 표시하지 않습니다.

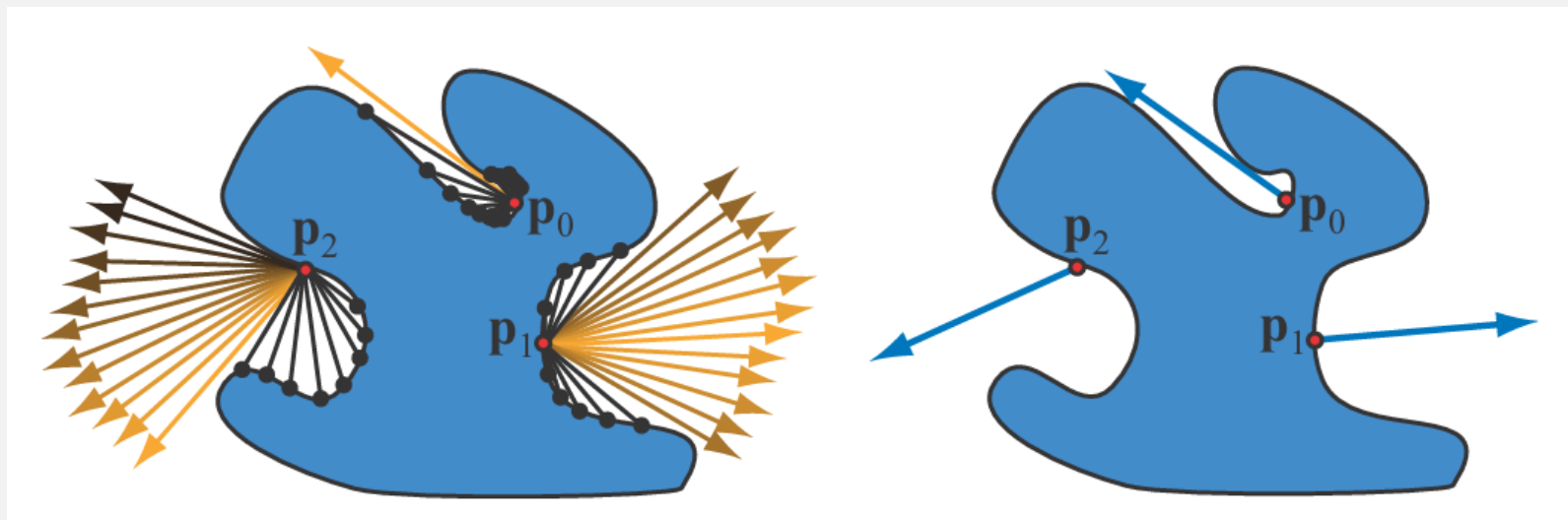


주변 폐색 이론 (Ambient Occlusion Theory)

주변 폐색 k_A 이외에도 Landis는 폐색되지 않은 방향의 코사인 가중 평균을 이용하여 구부러진 법선(bent normal)으로 알려진 폐색되지 않은 평균 방향도 계산하였습니다.

$$n_{bent} = \frac{\int_{l \in \Omega} l v(l) (n \cdot l)^+ dl}{\left\| \int_{l \in \Omega} l v(l) (n \cdot l)^+ dl \right\|} \quad * \text{수식 (11.10)}$$

Bent normal은 셰이더 처리에서 실제 geometr의 normal 보다 더 정확한 결과를 만들기 위해 사용될 수 있습니다. (실제 도달 가능한 라이트맵, irradiance volume, 그리고 이후 다룰 GI 결과들의 조화 등...)



* p0, p1, p2에서 뻗어나가는 파란색 선분의 방향이 bent normal, 실제 물체의 normal 방향과 다를 수 있다

주변 폐색 이론 (Ambient Occlusion Theory) - 요약

주변 폐색은 주변 형상의 영향으로 주변광을 얼마나 받을 수 있는지를 0~1로 나타낸 값이 됩니다.

아니 이렇게 수식이란 싸워서 얻은결론이
이거 저거 통치고 나면 빛이 얼마나 들어오는지 그냥 scalar하나랑 vector하나 남는다고

$$E(n) = k_A \pi L_A \quad * \text{수식 (11.9)}$$

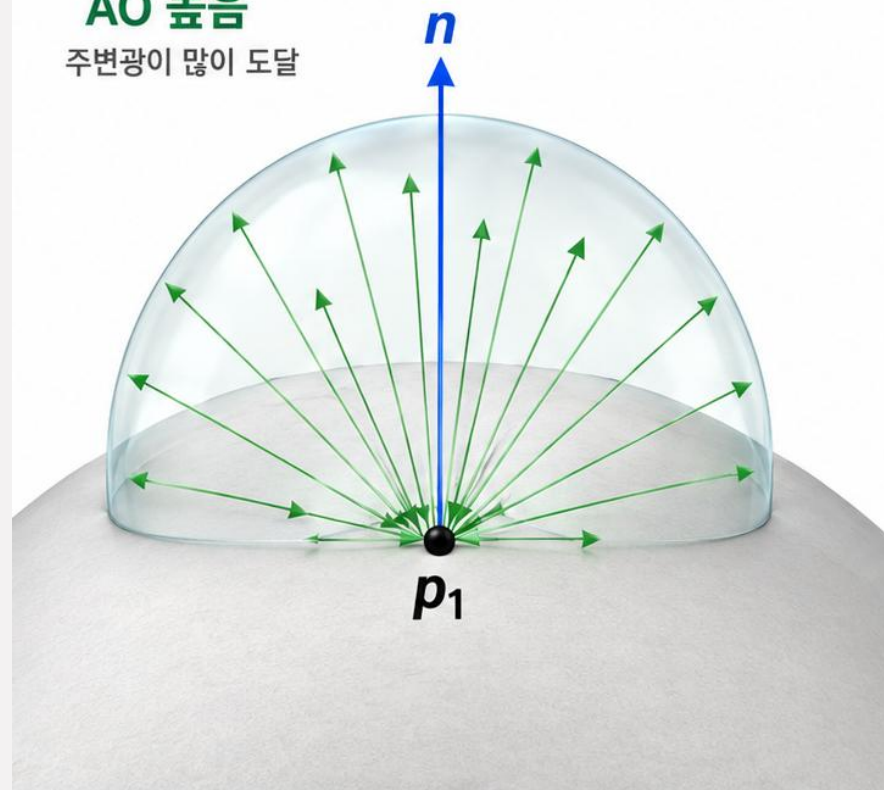
주변 폐색(AO)의 핵심 개념

점 p가 주변광을 얼마나 받을 수 있는지를 0~1로 나타낸 값

열린 표면

AO 높음

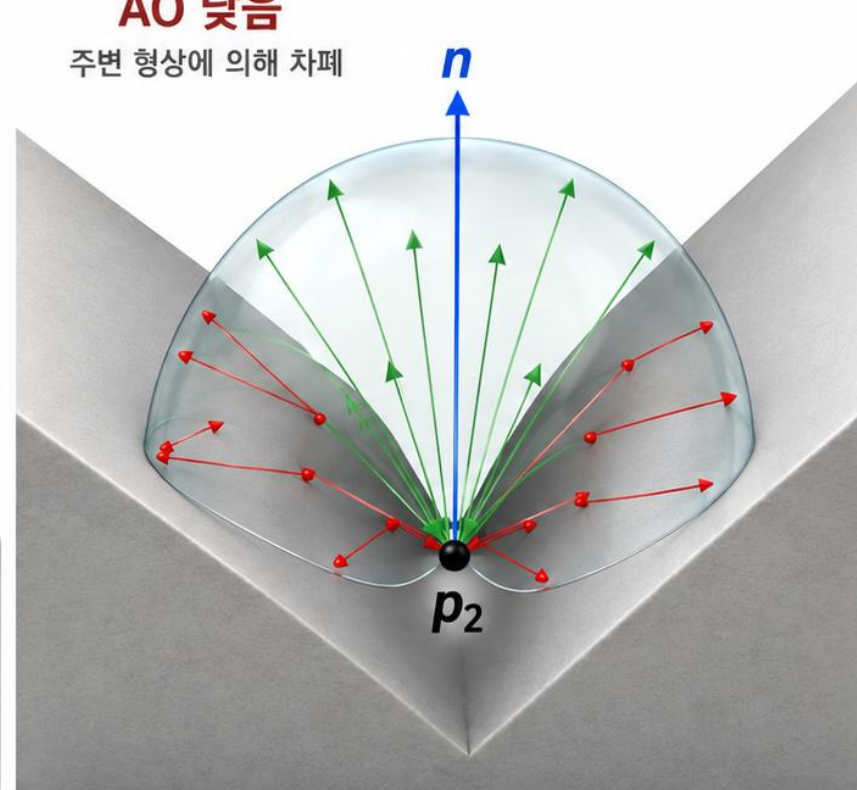
주변광이 많이 도달



오목한 홈(crease)

AO 낮음

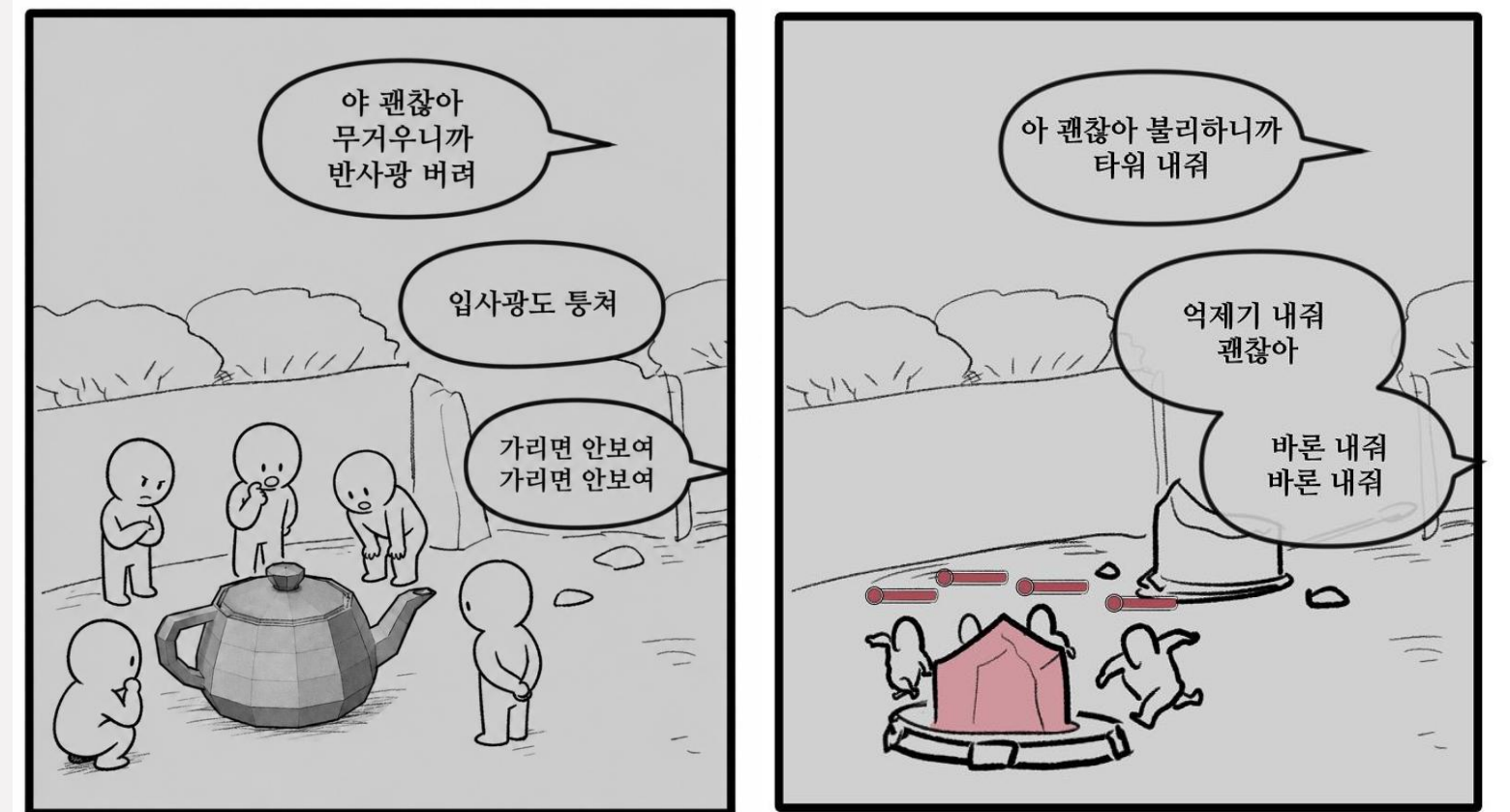
주변 형상에 의해 차폐



$$AO \approx \frac{\text{도달한 주변광}}{\text{원래 받을 수 있는 주변광}}$$

통치고 넘어간 가정 3개

표면은 Lambertian – (각도에 관계 없이 균일한 outgoing radiance)
Incoming radiance도 균일 (diffuse illumination만 존재)
차폐면 (또는 차폐 방향)에서 오는 radiance는 0



가시성과 차폐도(Visibility and Obscurance)

한글판은 Obscurance를 모호성이라고 번역했는데, 차폐도가 더 맞는 번역같습니다
뭐가 모호하다는건지 진짜 보면서 너무 모호했어...

주변 폐색 계수를 계산하기 위해서는 가시성 함수 $v(l)$ 가 필요했습니다.

$$k_A = \frac{1}{\pi} \int_{\Omega} v(l)(n \cdot l)^+ dl \quad \text{* 수식 (11.8), 주변 폐색}$$

오브젝트의 표면 p 에서 l 방향으로 ray를 쏘아서, 다른 물체에 닿게 되는지를 기반으로 $v(l)$ 을 정의하는건 간단하지만, 이는 닫힌 공간에서 결과가 좋지 못합니다.

다양한 오브젝트가 있는 닫힌 방에서는 표면에서 발사한 모든 광선은 무언가에 닿기 때문에 $v(l)$ 은 언제나 0이 되고, 자연스럽게 k_A 값도 0이 되는 문제가 생깁니다.

이런 닫힌 공간에서는 물리적인 가시성을 시뮬레이션 하지 않고 주변 폐색(AO)의 외형을 재현하려는 경험적인 접근 방법이 더 나은 결과를 내는 경우가 많습니다.

가시성과 차폐도(Visibility and Obscurance)

Miller의 접근성 셰이딩 개념에서 영감을 받아 Zhukov, Iones, Kronin는 가시성 함수 $v(l)$ 을 거리 매핑 함수 $p(l)$ 로 대체해, 주변 폐색 계산을 수정하는 obscurance라는 개념을 도입합니다.

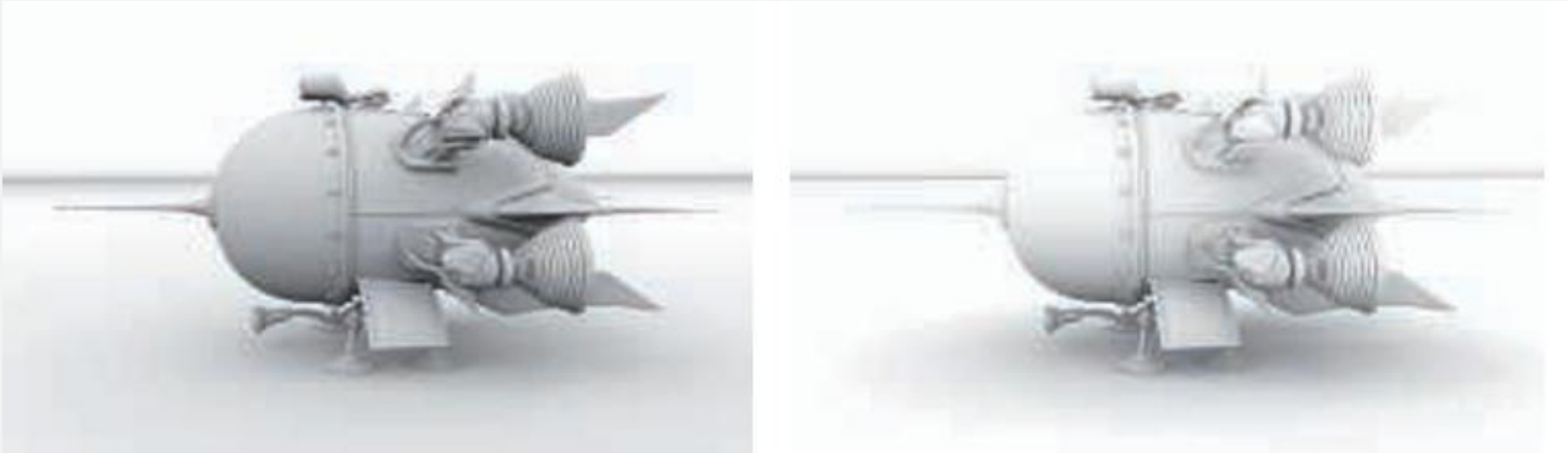
먼지가 쌓이거나, 변색된 표면은 주로 닳는 물체가 안 닿아서 생긴다는 내용
Accessibility를 표면p가 구형 probe에 얼마나 쉽게 닿을 수 있는가로 정의했다고 함

$$k_A = \frac{1}{\pi} \int_{\Omega} p(l)(n \cdot l)^+ dl$$

* 수식 (11.11), 주변 폐색 - 거리매핑 함수

두 가지 유효한 값 (Shadow때도 그랬듯이, 0과 1)이 있는 $v(l)$ 과 달리, $p(l)$ 은 표면에 도달하기 전까지 광선이 이동한 거리를 기반으로 하는 연속함수입니다. 교차 거리가 0일때 0이며, 미리 지정한 d_{max} 보다 먼 거리에서 교차하거나 아예 교차하지 않는 경우는 1이 됩니다. d_{max} 를 넘는 교차 거리는 검사할 필요가 없기 때문에, k_A 의 계산 속도를 상당히 높힐 수 있게 됩니다.

그림은 주변 폐색과 주변 차폐도(ambient obscurance)의 차이를 보여줍니다. 주변 폐색은 먼 거리에서도 교차점이 감지되어 k_A 값에 영향을 미치기 때문에, 상대적으로 더 어둡게 나옵니다.



가시성과 차폐도(Visibility and Obscurance)

Obscurance의 거리 감쇠를 실제 광전달 현상과 연결해 물리적으로 설명하려는 시도도 있었지만, 차폐 효과는 단순히 occluder까지의 거리만으로 결정되지 않으므로 Obscurance 자체는 물리적으로 정확한 모델이 아닙니다.

그럼에도 불구하고 보는 사람의 기대에 부합하는 그럴듯한 결과를 내는 경우가 많습니다. 한가지 단점은 $d_{\max}$ 값을 수작업으로 조정해야 한다는 것입니다.

이처럼 직접적인 물리적 근거는 없지만 지각적으로 설득력 있는(perceptually convincing) 기법을 통한 절충안은 컴퓨터 그래픽스에서 흔한 일입니다.

대부분의 목표는 믿을만한 이미지를 만드는 것이기 때문에, 문제될 것은 없습니다. 이론에 기반을 둔 방법의 장점은 자동으로 동작할 수 있고, 현실 세계에 대한 추론을 통해 더 개선할 수 있다는 점입니다.

* 앞으로 다룰 많은 파트도 통치고 넘어갔던 부분을 어떻게 살릴지(개선할지)에 대해서 꾸준히 다룹니다.



훈련소에서 물리 박사님에게 그래픽스의 빨간약을 전했을 때 세상 무너진 표정이었다.

뭐라고 수치해석도 모자라서 이런 구라들을 섞어 치고 있었다고

상호 반사의 반영 (Accounting for Interreflections)

AO로 생성된 결과는 시각적으로 설득력이 있지만, 완전한 전역 조명 모델(full global illumination)의 결과보다는 어둡게 나옵니다.

AO와 Full GI의 차이를 만드는 주요 원인은 상호 반사(interreflections)입니다. 이전에 식 11.8에서 차폐된 방향에서 들어오는 radiance를 0이라고 가정했지만, 실제로는 그 방향들에서도 상호 반사에 의해 0이 아닌 radiance가 들어오게 됩니다.

이 Full GI와의 차이는 k_A 값을 키움으로서 보정할 수 있습니다. 가시성 함수 대신 직전에 다룬 거리 매핑 함수를 사용하면, 차폐된 방향에 대해서도 0보다 큰 값을 가지는 경우가 많아 문제를 완화하는데에 도움이 됩니다.



상호 반사의 반영 (Accounting for Interreflections)

상호 반사를 조금 더 정확하게 추적하려면 Full GI에서 다루는 재귀적인 문제를 풀어야 하므로 비용이 많이 들게 됩니다.
 k_A 값을 계산하는 비용은 Full GI에 비해서 훨씬 저렴하지만, 과도하게 어두워지는 것을 막기 위해서는 누락된 빛을 어떤 형태로든 포함시키는 것이 좋습니다.

Stewart와 Langer는 AO가 가정했던 Lambertian surface와 diffuse illumination 환경에서는 한 점 p에서 보이는 주변 표면들의 radiance가 자신의 radiance와 비슷하다는 경향이 있다는 관찰에 기반하여 재귀 없이 해석적인 식을 얻어내었습니다. 책에서는 다루지 않았지만, 식이 유도되는 과정을 따라가봅시다.

기존 AO에서 차폐되지 않은 부분을 통해 받는 irradiance는 다음과 같습니다.

$$k_A \pi L_i \quad \text{* 수식 (11.9) 참고}$$

차폐 방향에서의 incoming radiance는 현재 점의 outgoing radiance와 비슷합니다 (같다고 가정)

$$L_{blocked} \approx L_o(p)$$

막힌 영역의 cosine-weighted 비율이 $1 - k_A$ 이기 때문에 차폐 방향에서 받은 irradiance는 다음과 같습니다.

$$\pi(1 - k_A)L_o$$

따라서 전체 irradiance는 이렇게 정리 할 수 있고

$$E = \pi k_A L_i + \pi(1 - k_A)L_o$$

$$L_o = \frac{\rho_{ss} E}{\pi} \quad \text{* Lambertian surface radiance}$$

Lambertian surface를 가정했기 때문에, outgoing radiance항을 다시 irradiance로 표현할 수 있습니다.

$$E = \pi k_A L_i + \pi(1 - k_A) \frac{\rho_{ss} E}{\pi}$$

해당 식을 정리하면 다음과 같은 식을 찾을 수 있습니다.

$$E = \pi k_A L_i + \rho_{ss}(1 - k_A)E$$

$$E = \frac{\pi k_A L_i}{1 - \rho_{ss}(1 - k_A)} \quad \text{* 수식 (11.12)}$$

$$E - \rho_{ss}(1 - k_A)E = \pi k_A L_i$$

$$E[1 - \rho_{ss}(1 - k_A)] = \pi k_A L_i$$

위 식을 기존 AO 공식의 형태로 변환하면 k_A 와 ρ_{ss} 값을 통해 상호 반사를 복원하여 밝아진 k'_A 값을 구할 수 있게 됩니다.

$$k'_A = \frac{k_A}{1 - \rho_{ss}(1 - k_A)} \quad \text{* 수식 (11.13)}$$

$$E = \pi k'_A L_i \quad \text{* 수식 (11.12)와 (11.13)을 합치면 기존의 AO와 동일한 형태의 식을 사용할 수 있다.}$$

상호 반사의 반영 (Accounting for Interreflections)

이렇게 유도된 결과는 상호 반사를 포함한 Full GI의 결과에 시각적으로 더 가깝게 만들어 주지만, ρ_{ss} (subsurface albedo)값에 크게 의존적입니다.

* 물론 실제 Color Bleeding처럼, 주변 표면의 색상이 실제로 반영되진 않습니다.
* 같은 albedo값이라고 통쳤으니깐요

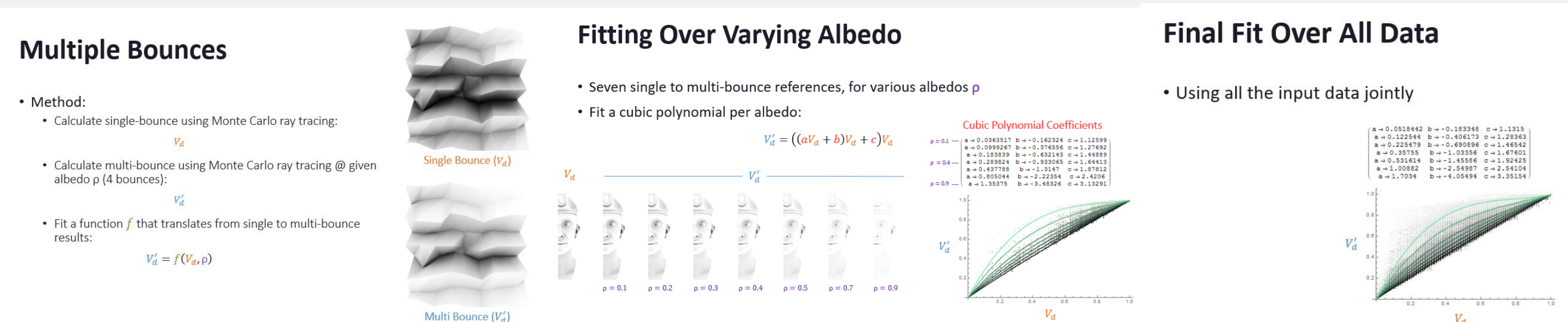
이 과정은 표면의 색상과 인접한 표면이 동일하다고 가정하여 작동하게 되며, 그렇기에 결과가 color bleeding과 비슷한 원리로 밝아지게 됩니다.

Hoffman과 Mitchell은 이 방법을 사용해 지형(terrain)에 하늘 전체에서 오는 확산 조명(diffuse lighting) 정보로 AO 정보를 미리 구워두었습니다. 이는 시간에 따라 sky color/radiance가 변경되더라도 GI를 다시 계산할 필요 없이, 변경된 값만 곱해주면 되게 되었습니다.

Jimenez는 주변 표면의 albedo가 크게 다르지 않고, diffuse illumination 환경에서 path tracing한 정보들을 모아 3차 다항식의 계수를 튜닝하였습니다. k_A 와 ρ_{ss} 값만으로 k'_A 값을 예측할 수 있는 함수를 만들었다는 내용으로 이해하시면 될 것 같습니다.

Stewart-Langer의 방법은 해석적인 식을 통해 근사했지만, Jimenez는 실제 렌더링 환경의 path tracing정보를 기반으로 polynomial에 압축해두었기 때문에 물리적으로 올바름과 별개로 path tracing 결과와 평균적으로 더 유사하게 k'_A 값을 유추해내게 되었습니다.

*Jimenez의 방법은 블리자드 소속 인원 4명이 SIGGRAPH 2016에서 발표한 내용이네요 [\(출처\)](#)



미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO 계수의 계산은 시간이 많이 소요될 수 있어서, 렌더링에 앞서 오프라인으로 수행하는 경우가 많습니다. 엠비언트 오클루전을 비롯해 조명 관련 정보를 사전에 계산해두는 과정을 흔히 베이킹(baking)이라고 부릅니다.

AO를 사전 계산하는 가장 일반적인 방법은 몬테카를로 기법입니다. 1차시 RayTracing에서 짚고 넘어갔듯이, 반구 방향으로 광선을 쏘아 오브젝트의 차폐 여부를 검사하고, AO 계수를 수치적으로 계산합니다.

예를 들자면 법선 n 주위의 반구에 균일하게 분포된 N 개의 무작위 방향 l 을 고른 뒤, 그 방향으로 광선을 추적한다고 해봅시다. RayCast 결과를 바탕으로 가시성 함수 v 를 평가하면 AO는 다음과 같이 계산됩니다.

$$k_A = \frac{1}{N} \sum_{i=1}^N v(l_i)(n \cdot l_i)^+ \quad \text{* 수식 (11.14), 몬테 카를로 방법 가시성 주변 폐색} \quad k_A = \frac{1}{\pi} \int_{\Omega} v(l)(n \cdot l)^+ dl \quad \text{* 참고용 수식 (11.8), 주변 폐색}$$

주변 차폐도(Ambient Obscurance)를 계산할 때는 광선의 거리를 최대치로 제한할 수 있으며, v 의 값은 검출된 교차 거리를 기준으로 결정됩니다. (11.3.2 내용)

AO 계수의 계산에는 코사인 가중 인자가 포함됩니다 (각도에 따라 중요도가 다르다) 수식 11.14처럼 이를 직접 곱해서 넣을수도 있지만, 더 효율적으로 반영하는 방법은 중요도 샘플링(importance sampling)을 이용하는 것입니다. 반구 전체에 균일하게 광선을 쏘고 $n \cdot l$ 을 곱하는 대신 광선 방향의 분포 자체를 normal vector 방향에 가까운 쪽으로 많이 쏘아지게 하는 것입니다. 이런 샘플링 기법을 Mally's method라고 합니다.

이러한 AO의 사전 계산은 CPU / GPU에서 수행할 수 있다. 두 경우 모두 복잡한 geometry에 대한 raycast를 가속하는 라이브러리를 사용할 수 있으며 가장 널리 쓰이는것은 Embree(CPU), OptiX(GPU)입니다.

과거에는 깊이 맵(depth map)이나 오클루전 쿼리 같은 GPU 파이프라인의 결과물을 AO계산에 사용하기도 했으나, GPU에서 동작하는 RayCast 솔루션이 널리 보급되며 오늘날에는 그런 방식을 쓰는 경우가 줄었습니다.

상용 모델링/렌더링 소프트웨어 패키지는 대부분 AO를 사전 계산하는 기능을 제공하고 있습니다.

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO는 표면의 모든 위치 p마다 값이 다르지만, 무한히 많은 p에 대한 모든 정보를 저장할 수는 없습니다. 이를 해결하기 위해 일반적으로 Texture, Volume, Mesh의 Vertex에 저장하고 런타임에는 이를 복원하는 과정을 거치게 됩니다. 저장 방식에 따른 특성과 문제점은 신호의 종류와 무관하게 대체로 비슷합니다. 이번 장에서 다루는 AO를 포함해서 다음장에서 다룰 방향성 차폐(DO), 사전계산된 조명등의 정보는 동일한 방법론을 적용할 수 있고, 3차시에 설명하게 될 예정입니다.

사전 계산된 데이터는 오브젝트들이 서로에게 미치는 AO효과를 모델링하는데에도 사용할 수 있습니다.

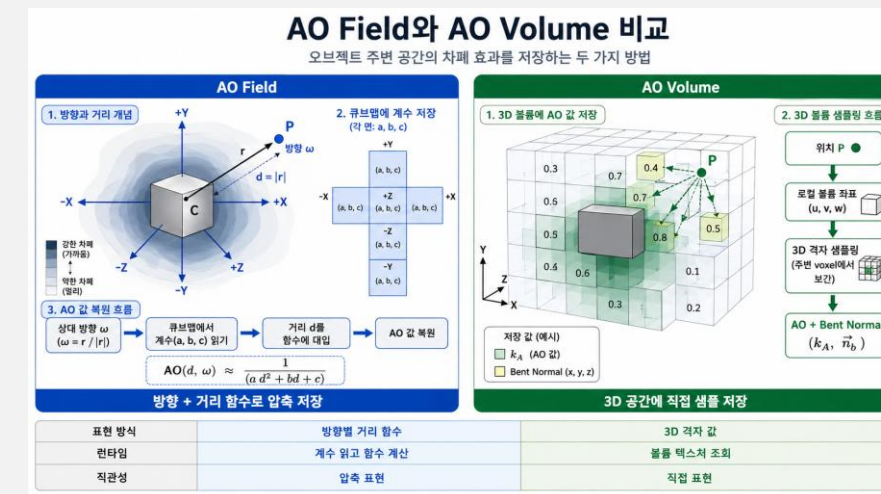
Kontkanen과 Laine는 오브젝트로부터의 거리에 따른 AO 값의 감쇠를 2차 다항식의 역수로 근사하고, 방향 ω 별 계수를 큐브 맵에 저장하는 Ambient Occlusion Field를 제안하였고,

Malmer등은 AO계수와 bent normal(선택)을 3D Grid상에 저장하는 Ambient Occlusion Volume을 제안하였습니다.

동적인 AO처리가 무거우니 오브젝트단위로라도 구워둘 수 있을까?를 고민한 흔적이고 현대 AO는 대부분 이렇게 처리하지 않아서 넘어갑니다

*게임 엔진에서 이런 방식으로 처리하는거 본적도 없기도 하고, 비슷한 이름의 기능들은 현대 AO처리 방식에 더 가까웠음
*2000년 후~ 2010년대 상용 게임의 커스텀 엔진에서 동적인 AO처리를 위한 기능들이었다 보시면 될 듯

*물론 뒤에 다루겠지만, 동적인 오브젝트의 AO를 정말 싸게 처리해야하는 상황이 있습니다. 모바일 기기에서 동적인 오브젝트에 꼭 필요한 AO를 구현할 때 같은...



미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO 값을 저장하는 방법으로 어떤것을 선택하더라도, 다루는 대상이 연속 신호라는 점을 유념해야 합니다.
공간상의 특정 지점에서 Ray를 쏘는것은 샘플링이고, 그 결과로부터 셰이딩에 사용할 값을 보간하는것은 복원입니다.

Kavan등은 최소 제곱 베이킹이라는 방법을 제안했습니다.

Mesh 전체에 균일하게 AO 계수를 샘플링하고, 보간된 값과 샘플링된 값의 차의 제곱의 합이 최대한 작게 만드는 값을 선정해 계수를 저장하게 합니다.

*분산이나 표준편차가 가진 아이디어랑 비슷하고, 이것을 최소화 하는 값을 만들어 저장하게 한다는 아이디어

예시로는 Mesh의 Vertex를 들었지만, Texture나 Volume에 해당 값을 저장할때도 동일한 방법론을 사용할 수 있습니다.

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

Destiny₍₂₀₁₄₎는 PS3₍₂₀₀₆₎에서 PS4_(2013말) / XBOX360₍₂₀₀₅₎에서 XBOX ONE_(2013말)으로 넘어가는 과도기에 출시되어 신규 플랫폼에 기대되는 높은 품질과 구세대 플랫폼의 성능-메모리 제약 사이의 균형잡힌 솔루션이 필요했습니다.

게임에는 동적으로 시간이 변하는 솔루션이 있어, 어떤 사전 계산 방식을 사용하더라도 이를 올바르게 반영할 수 있어야 했습니다.

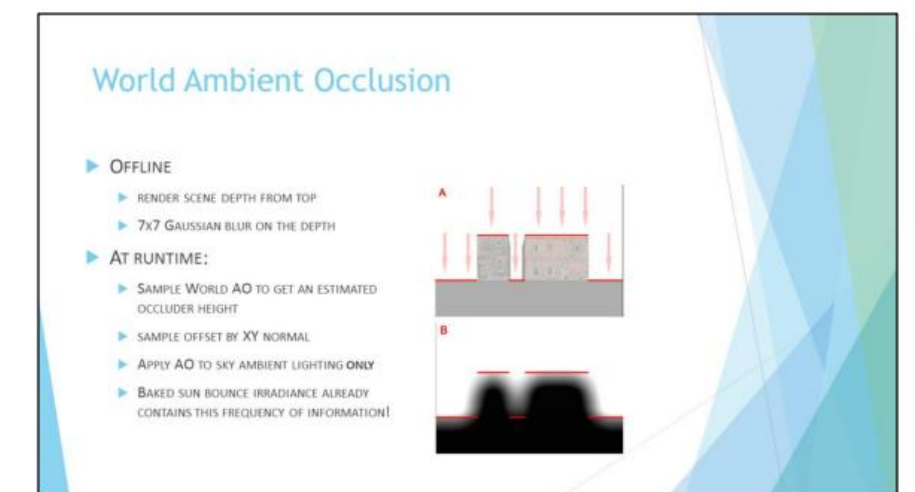
개발진은 그럴듯한 외형과 낮은 런타임 처리 비용을 이유로 AO를 선택했으며 AO는 가시성 계산을 조명과 분리하여 시간대와 무관한 사전 데이터를 재사용할 수 있었습니다.

Ubisoft의 어세신크리드와 파크라이 시리즈도 간접 조명 솔루션을 강화하고자 사전 계산된 AO 형태를 사용했습니다. 월드를 위에서 내려다보는 시점에서 렌더링하여 깊이 맵을 여러 휴리스틱을 통해 월드 공간 AO맵을 만들어 각 오브젝트의 월드 공간 위치를 텍스처 공간으로 투영하는 방식으로 모든 오브젝트에 AO를 적용했습니다. (Ubisoft는 이를 World AO라고 부름)

*Digital Dragons 2014 유비소프트 발표 자료 [\(출처\)](#)



Looks of "World ambient occlusion" in Assassin's Creed 3



Algorithm very approximate and requires artists to tweak some "magic values", but extremely cheap to evaluate and quite robust.
...As long as your scene can be approximated by heightfield!

주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

정적인 장면의 경우 AO 계수와 bent normal을 사전 계산할 수 있습니다.

하지만 오브젝트가 움직이거나 형태가 바뀌는 장면에서는 이런 계수들을 실시간으로 계산하는 편이 더 나은 결과를 만들어냅니다.

이를 위한 방법은 크게 오브젝트 공간(object space)에서 동작하는 것과, 스크린 공간(screen space)에서 동작하는 것으로 나눌 수 있습니다.

11.3.5에서는 주로 object/world space에서 동작하는 AO를 다룰 예정입니다.

AO를 계산하는 오프라인 방식은 표면에서 장면을 향해 수십에서 수백개에 이르는 다량의 광선을 쏘고 교차 여부를 검사하는 과정을 포함합니다.

이는 비용이 큰 연산이기 때문에 실시간 기법들은 이 계산의 상당 부분을 근사하거나 피해가는 방법에 초점을 맞춥니다.

Bunnell은 표면을 Mesh Vertex에 배치된 원반(Disk)모양 요소들의 집합으로 모델링 하여 AO 계수와 bentnormal을 계산합니다.

원반을 택하면 한 원반이 다른 원반을 가리는 이유를 해석적으로 계산할 수 있어 ray를 쏘 필요가 없어지기 때문입니다.

하지만 단순히 모든 Disk의 폐색 계수를 합산하면 이미 가려진 부분을 새로 가리는 문제가 발생할 수 있습니다.

Bunnell은 이 문제를 2Pass로 처리해서, 두번째 Pass에서는 첫번째 Pass에서 계산한 AO계수에 따라 다른 Disk (Surface)에 주는 영향을 줄이도록 하여 해결했습니다.

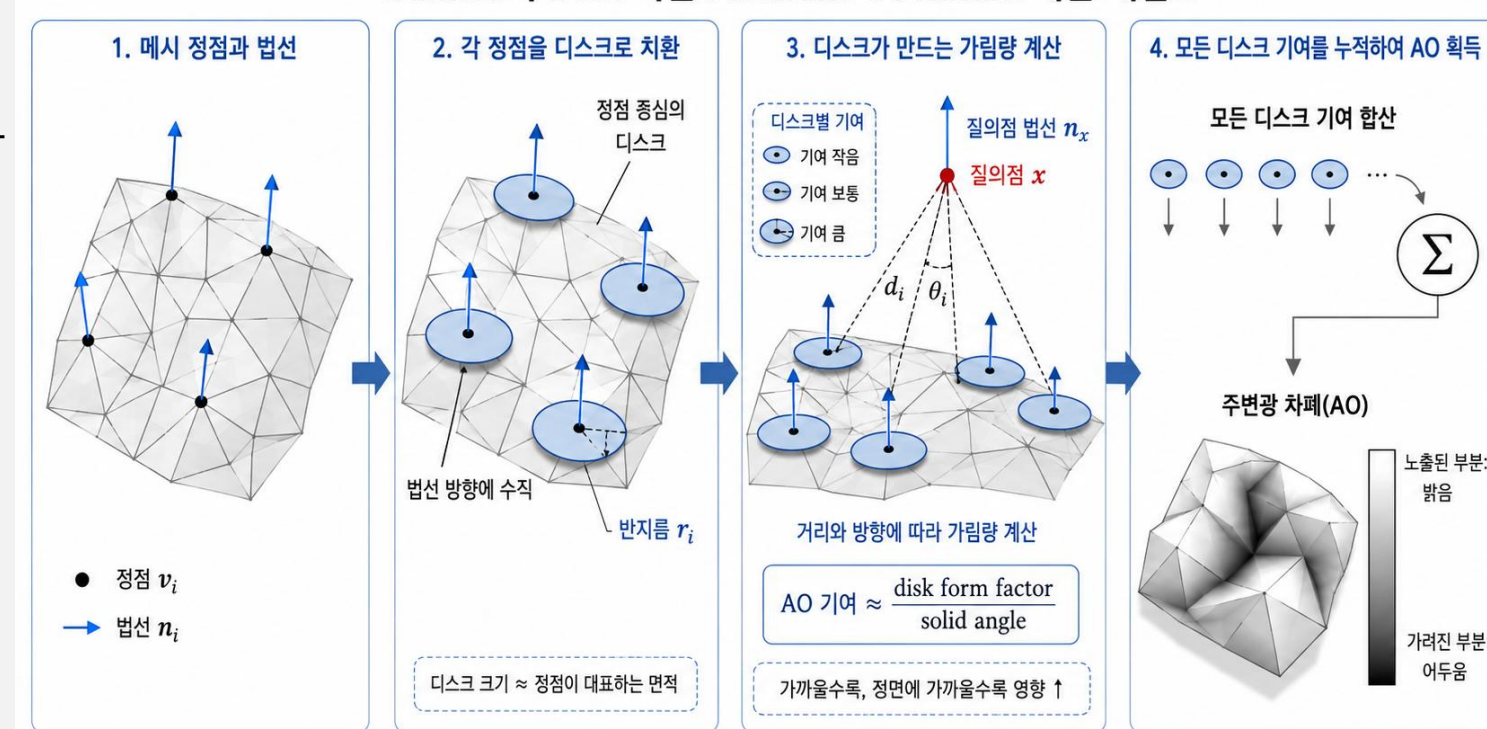
각 Disk사이의 폐색을 계산하는것은 $O(n^2)$ 연산으로, 비용이 많이 나가게 됩니다.

이것은 Disk를 hierarchical tree로 구성하여 서로 먼 표면에 있을 때는 parent 하나로 처리해 실질적으로 $O(n \log n)$ 까지 처리 비용을 낮추게 됩니다.

해당 기술은 영화 캐리비안의 해적의 최종 렌더링에 사용되었습니다.

Hoberock은 11.3.2에서 다룬 Obsurance와 유사한 거리 감쇠 계수를 포함해 Bunnell의 알고리즘을 조금 더 비싸지만 고품질로 수정하였습니다.

Bunnell의 Disk 기반 Ambient Occlusion 계산 개념도



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

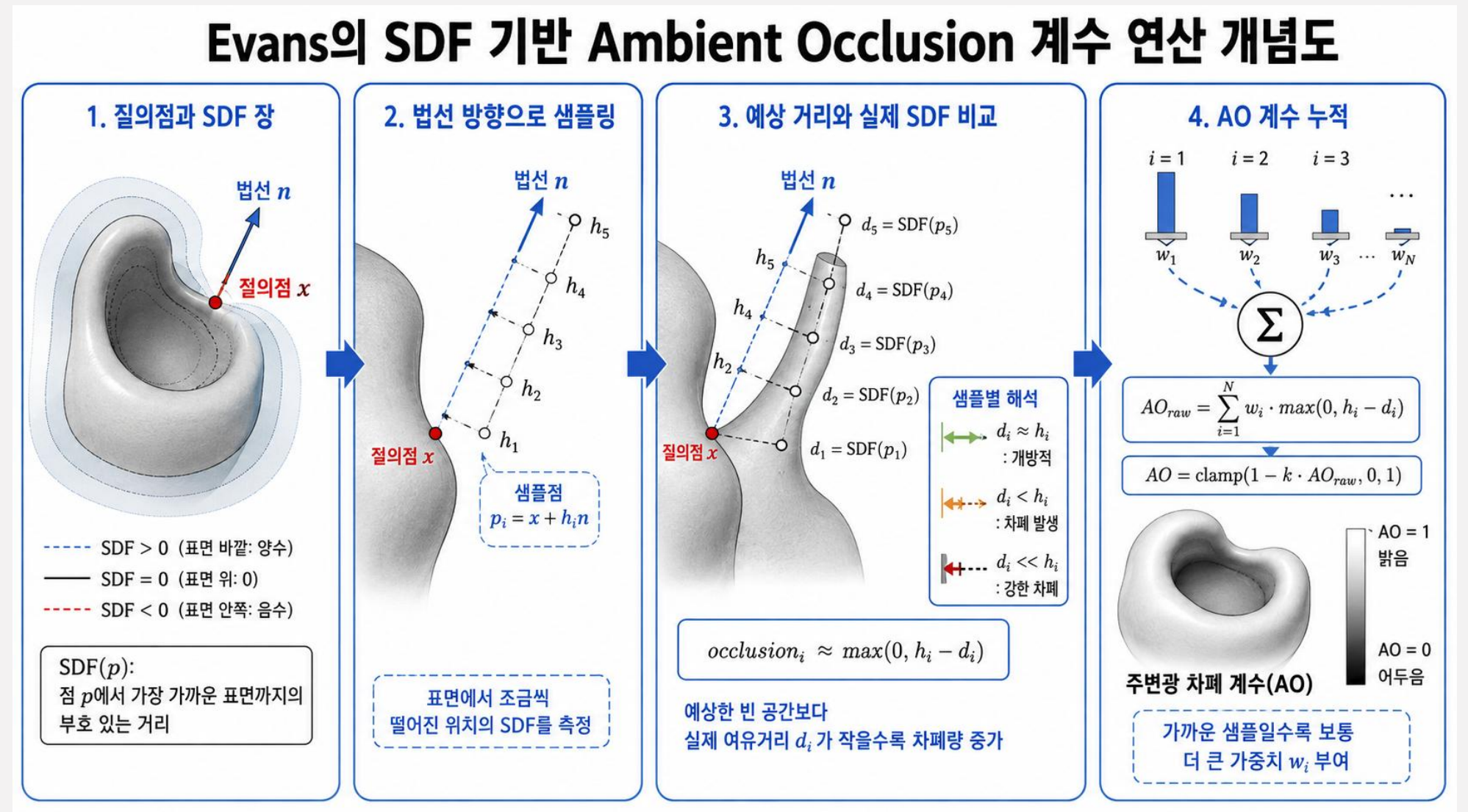
부호 있는 거리 필드 (SDF, Signed Distance Field)는 object 내부 깊숙할수록 음수, object의 표면(surface) 지점에서는 0, object 외부에서 멀어질수록 양수가 되는 함수입니다.

AO에서는 SDF를 읽었을때, 해당 지점에서 가장 가까운 geometry의 존재를 검사할 수 있게 되기 때문에 사용합니다.

Evans는 이 특징을 살려 기존의 hemisphere 방향을 검사하던 AO대신 normal방향으로 점점 먼 점에 대해 SDF를 sampling하는 방법을 제안하였습니다.

이는 SDF값이 3차원 텍스처 (Volume)에 저장되는 대신 해석적으로 표현하는 경우에도 같은 접근법을 쓸 수 있습니다.

이 방법은 물리적이지는 않지만, 시각적으로 만족스러운 결과를 만들 수 있습니다.



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

SDF를 AO에 사용하는 방법은 Wirght에 의해 한층 더 발전했습니다.
휴리스틱으로 normal 방향의 SDF를 샘플링하여 AO 계수를 계산하는 대신 Cone tracing을 수행합니다.

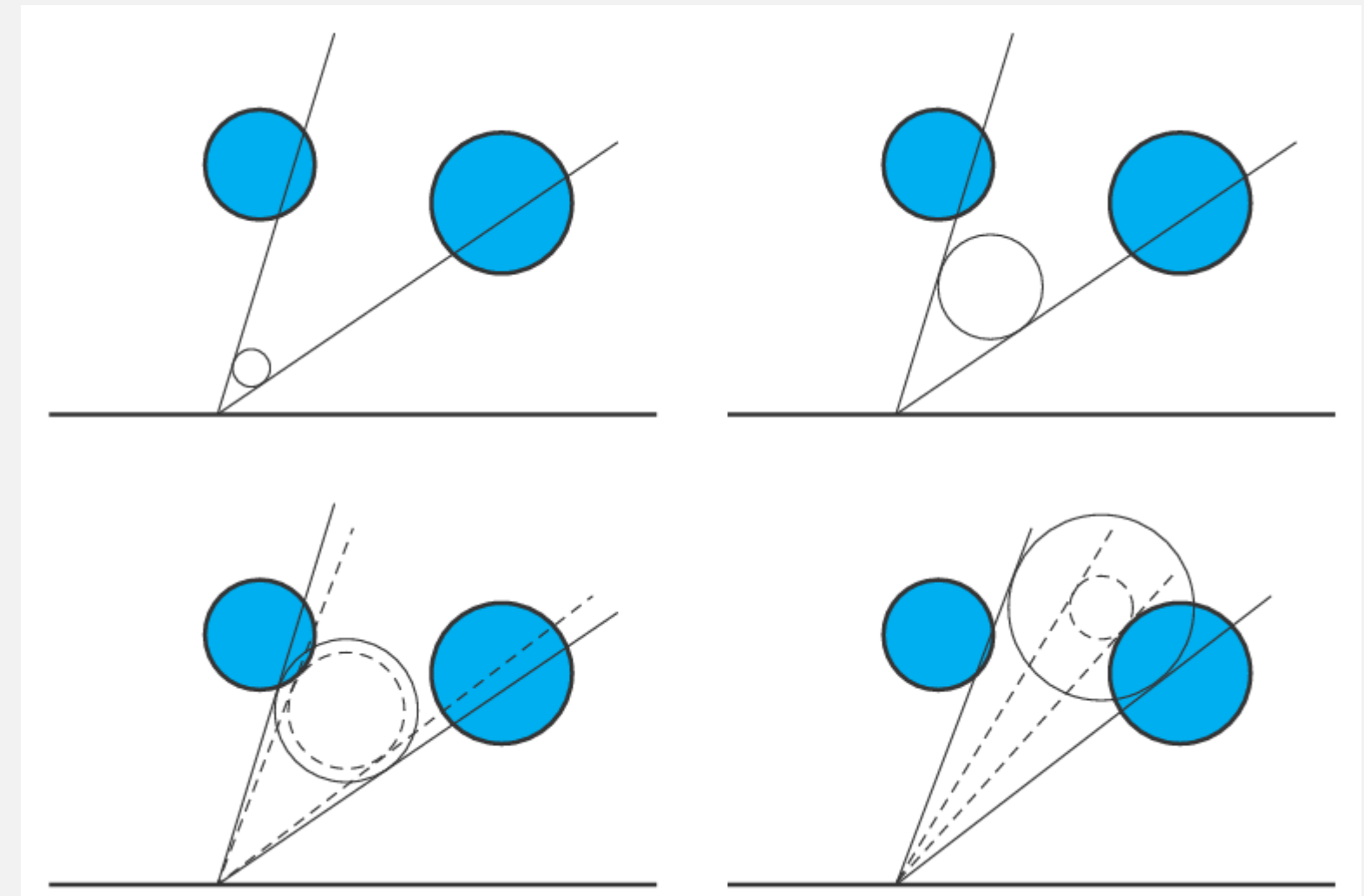
법선 기준 반구에 약 9개의 cone을 배치하고
각 cone 축을 따라 SDF sphere / cone steeping을 수행합니다.
하나의 cone내에서는 값을 합산하는게 아니라 minimum visibility를 유지하도록 합니다.

9개의 cone의 visibilit를 결합하여 AO계수와 bentnormal을 생성합니다.

SDF를 통해 빛이 실제로 통과할 수 있는 가능성을 근사하고 있었다 보심 될 듯

장면에 대한 전역 SDF뿐 아니라 개별 object나
논리적인 object group의 SDF도 함께 사용하여
정밀도를 높였습니다.

*동적 AO에서 SDF를 사용한다는 것은 SDF를 매 프레임 생성한다는 뜻이 아닙니다.
일반적으로 geometry의 distance-field representation은 미리 준비하고,
런타임에는 현재 shading point와 object transform에 따라 SDF를 query하여 AO를 동적으로 계산합니다



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

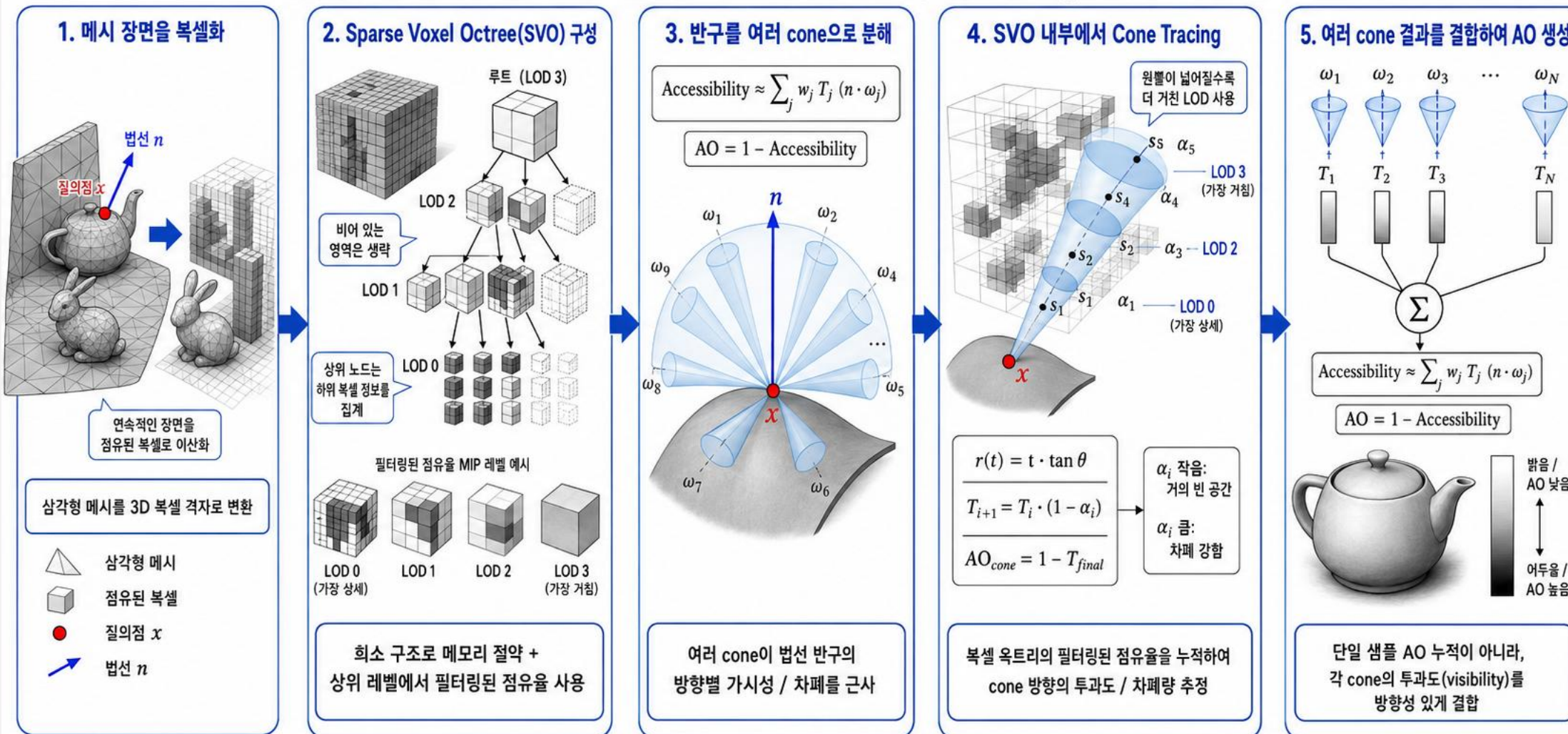
Crassin은 장면을 먼저 voxel로 변환합니다.

거대한 3D 공간을 dense volume으로 저장하면 너무 크기 때문에, 빈 공간은 크게 묶고 geometry가 있는 곳만 세분화 하는 Sparse Voxel Octree(SVO)를 구성합니다.

Octree를 구성하였기 때문에, Cone Tracing에서 떨어진 cone은 상위 레벨의 octree를 참조할 수 있게 됩니다.

Crassin의 Sparse Voxel Octree 기반 Cone Tracing AO 계산 개념도

핵심: 장면을 SDF가 아니라 희소 복셀 옥트리(SVO)로 표현하고, cone tracing으로 방향별 차폐를 적분



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

Ren은 AO가 흐릿하다는 점을 이용해 Occulsion Geometry를 구 모음으로 근사했습니다.

단일 구에 가려지는 표면점을 표현하는 가시성 함수를 구면 조화 함수를 사용해 표현할 수 있고 여러 개의 구에 의한 차폐의 결과는 개별 구의 가시성 함수를 곱한 결과가 됩니다.

하지만 구면조화 함수끼리의 곱을 계산하는것은 비용이 큰 연산이고 이를 해결하기 위해 개별 구면 조화 함수의 로그를 더한 뒤, 그 결과에 지수를 취하는 것입니다. 이렇게 하면 가시성 함수를 곱한것과 동일한 최종 결과를 얻으면서도, 훨씬 저렴하게 처리할 수 있습니다.

$$\log \prod_{i=1}^N V_i(l) = \sum_{i=1}^N \log V_i(l)$$

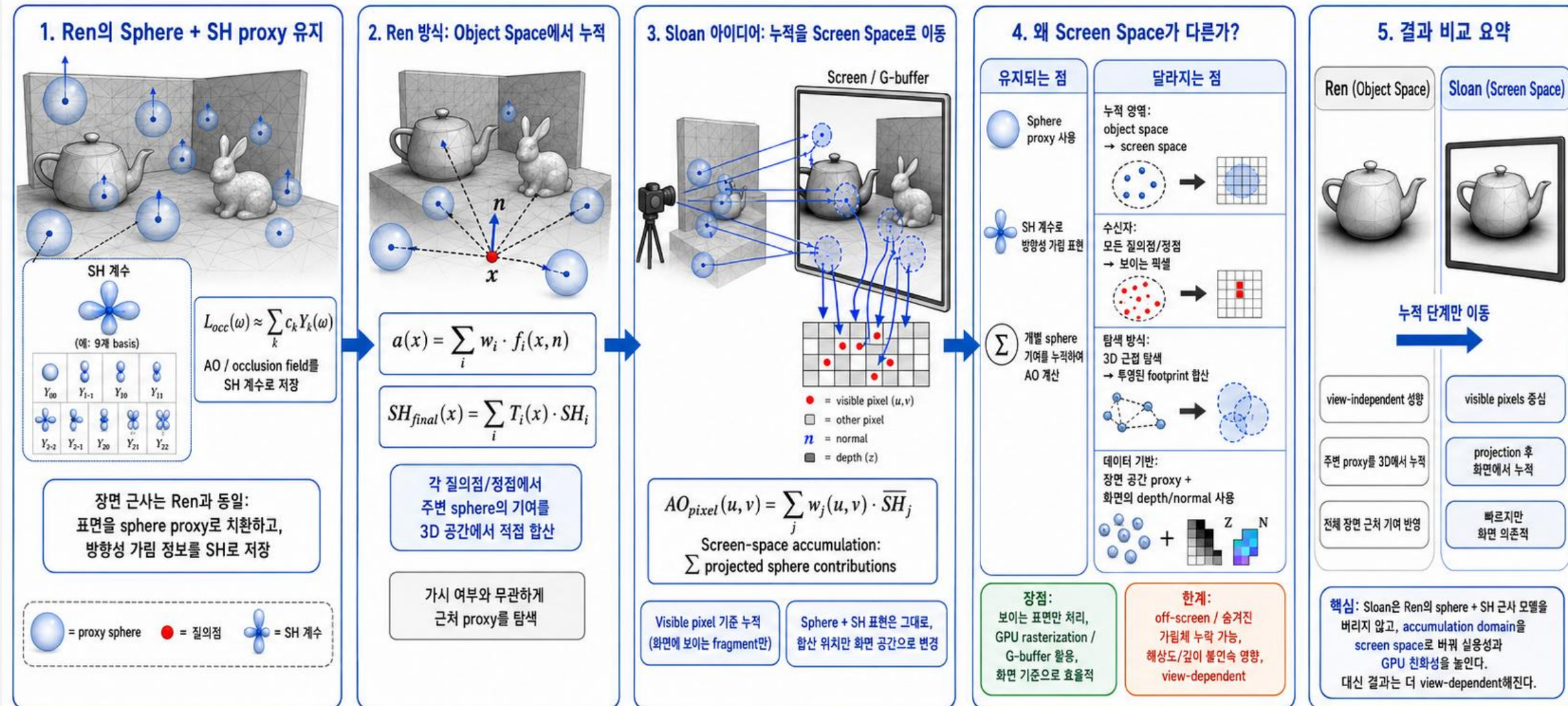


주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

Sloan은 Ren의 Sphere + SH방식의 visibility 누적 연산을 screen space에서 처리하는 방법을 제안하였고, Hill은 SH 함수를 2차 계수까지만 저장하여 성능이 제한적인 하드웨어에서도 사용할 수 있지만, 흐릿한 AO 처리만 가능한 방법을 제안했습니다.

Sloan의 Screen-Space Sphere + SH AO 개념도

Ren의 Sphere + SH proxy는 유지하되, AO 계수 누적을 object space에서 screen space로 이동



기호: x (질의점), n (법선), SH_i (i 번째 sphere의 SH 계수), $T_i(x)$ (전달/가시 함수), w (가중치)
SH (Spherical Harmonics), AO (Ambient Occlusion)

스크린 공간 기법(Screen-Space Methods)

직전까지 다룬 object-space를 기반으로 만들어지는 AO들은 scene geometry를 계산해야 하기 때문에 구성된 씬의 복잡도에 따라 영향을 받게 됩니다. (안보이는 물체들도 연산 대상이 됩니다)

하지만 차폐에 대한 정보 중 일부는 깊이(Depth)나 법선(normal)처럼 이미 확보되어있는 스크린 공간 데이터 만으로도 유추해 낼 수 있습니다.

이러한 기법들은 장면이 얼마나 정밀한지와 무관하게, 오직 렌더링에 사용하는 해상도에만 좌우되는 일정한 비용을 갖습니다.
(이전 파트에서 Sloan의 방법이 가지는 메리트, 하지만 여기부터는 AO계수 누적을 위한 샘플링도 ScreenSpace에서 처리합니다.)

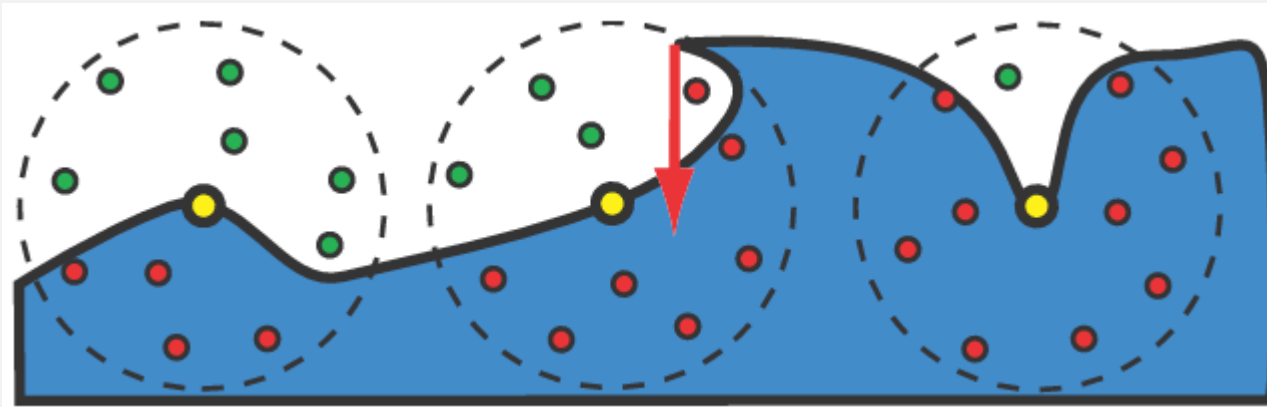
화면에 보이지 않는 geometry 정보는 알 수 없게 되는 문제가 있지만,
기존 렌더링 과정중에 남는 데이터거나, 렌더링 과정중에 조회한 데이터를 별도 버퍼에 저장해두기 때문에 GPU 친화적인 장점이 있습니다.

스크린 공간 기법(Screen-Space Methods)

Crytek은 Crysis에 사용된 동적 스크린 공간 **엠비언트 오클루전** (screen-space ambient occlusion, **SSAO**) 기법을 개발했습니다.

이들은 Z-Buffer만을 입력으로 사용하는 전체 화면 패스에서 AO를 계산합니다.

각 pixel의 페색 계수 k_A 는 픽셀 위치를 중심으로 한 구 내부에 분포시킨 점들의 집합을 z-buffer와 비교 검사하며 추정합니다.
 k_A 의 값은 현재 pixel의 z-buffer 값보다 앞에 있는 샘플의 개수에 대한 함수이다.
이를 통과하는 샘플수가 적을수록 k_A 값이 낮아지게 된다.



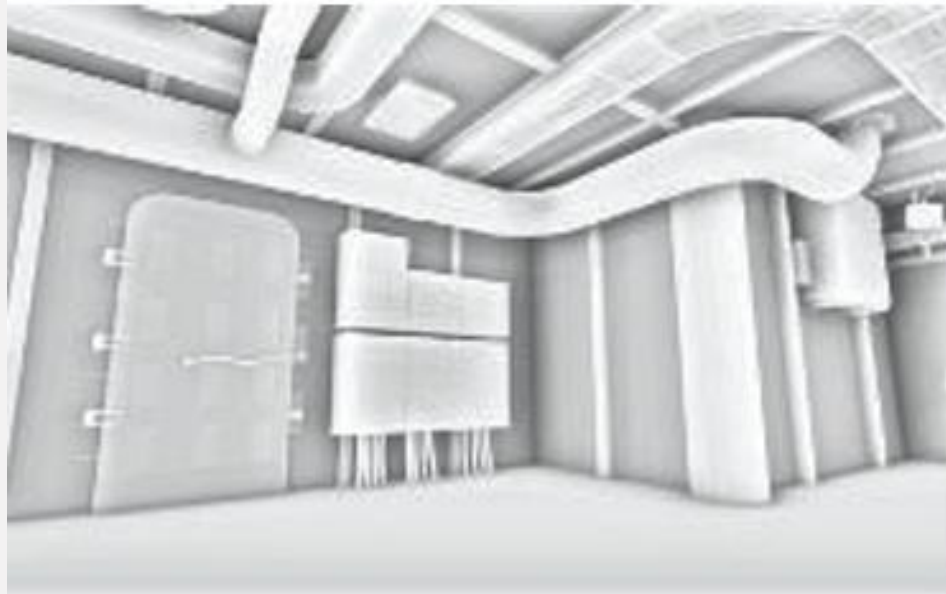
샘플들은 Obscurance factor와 마찬가지로 픽셀로부터 거리에 따라 감소하는 가중치를 가집니다.

스크린 공간 기법(Screen-Space Methods)

여기서 샘플에 $(n \cdot l) +$ 인자로 가중치를 주지 않기 때문에, 결과로 나타나는 AO는 부정확하다는 점에 유의해야한다.
표면 위쪽 반구(surface hemisphere)대신 전체 sphere를 대상으로 하기 때문에, 평평한 면도 조금 더 어두워지고
Edge는 주변보다 밝아지는 현상이 생깁니다.

하지만 그럼에도 결과물은 시각적으로 만족스러운 경우가 많습니다.

*SSAO Only



*Albedo



*SSAO + Albedo



*SSAO + Albedo + Shadow + Specular



스크린 공간 기법(Screen-Space Methods)

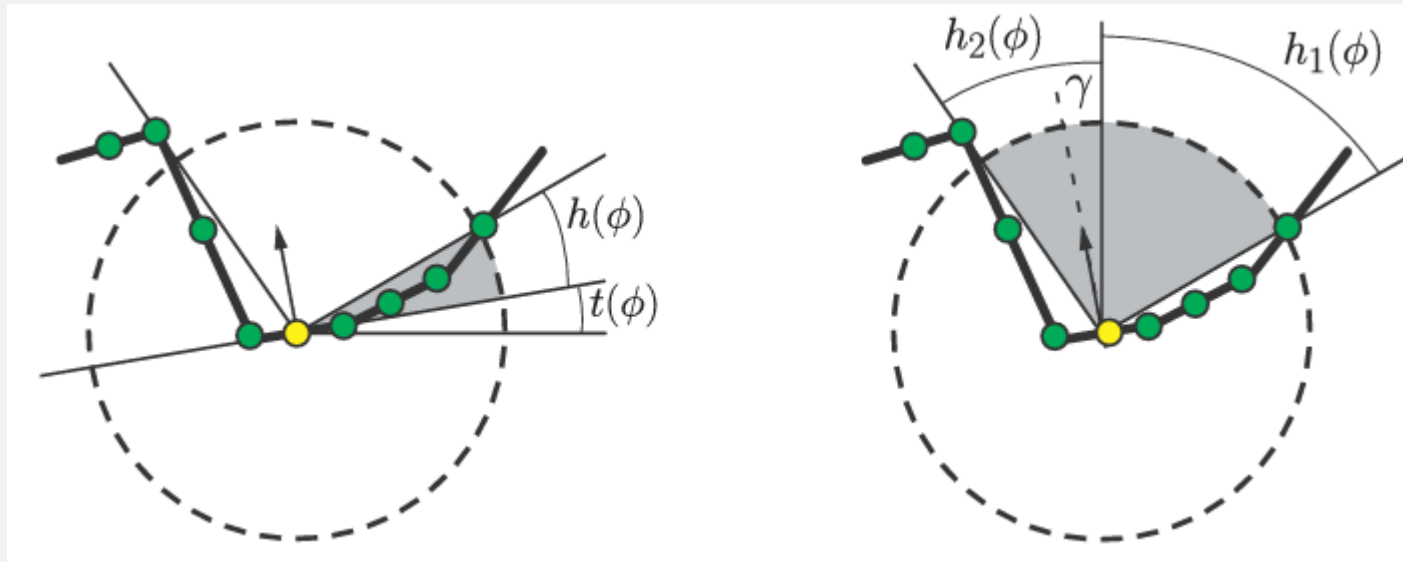
HBAO

Depth buffer를 height field로 보자

한방향으로 depth들을 보면서 가장 높은 depth를 가진 포지션을 보고 horizon angle h 를 구하기

여러 방향으로 위 과정을 수행해 막힌 각도 영역을 적분하기

이를 수행할 때 기존 AO의 cosine term 대신 거리 기반 감쇠 기반으로 AO값을 계산함



스크린 공간 기법 (Screen-Space Methods)

GTAO

HBAO가 가지는 아이디어에서 horizon 정보를 가지고 더 올바른 AO factor를 찾도록 만들었음

스크린 공간 기법(Screen-Space Methods)

RTAO

이제 하드웨어에 레이트레이싱을 위한 전용 장치들이 있으니
Depth버퍼에서 찾지 말고 RayTrace하죠? 라는 내용

스크린 공간 기법(Screen-Space Methods)

이런 스크린 공간 기법들은 대부분 주어진 점 주변 지오메트리의 단순화된 모델을 만들기 위해 z-버퍼를 반복적으로 샘플링하는 데 의존합니다. 실험에 따르면 높은 시각적 품질을 얻기 위해서는 수백개의 샘플이 필요하지만, 실시간 렌더링에서는 기기 사양에 따라 10~20개, 종종 그보다도 적은 수의 샘플이 작동할 수 있습니다.

이론과 실무 사이의 간극을 메우기 위해 스크린 공간 기법들은 일종의 공간적 디더링을 사용합니다. 화면의 픽셀마다 조금씩 다른 (회전시키거나 이동시킨) 무작위 샘플 집합을 사용하는 것입니다.

적은 샘플로 이루어진 AO계단의 주 단계가 끝나면, 전체 화면 필터링 패스를 수행합니다.

표면의 불연속면을 가로질러 필터링하는것을 막고 선명한 경계를 보존하기 위해 결합 양방향 필터링(joint bilateral filtering)을 사용합니다. 이에 대해서는 12.1.1에서 다룰 예정입니다.

이 필터는 깊이나 normal정보를 활용해 동일한 표면에 속한 샘플만 사용하도록 필터링을 제한합니다.

스크린 공간 기법(Screen-Space Methods)

또한 AO계산은 시간에 걸쳐 슈퍼샘플링 되는 경우도 많습니다.

매 프레임마다 다른 샘플링 패턴을 적용하고 차폐 계수를 지수 평균하는 방식으로 이루어집니다.
이전 프레임들의 z-buffer, moving vector, 카메라 변환등의 정보를 이용해 현재 프레임의 pixel위치와 맞추고
신뢰할 수 없는 데이터의 경우 폐기합니다.

이 데이터를 만들때는 depth, normal, moving vector등에 기반한 휴리스틱이 많이 (정말 많이!) 사용됩니다.

AO에서는 이것이 치명적인 아티팩트를 만드는 경우가 드물지만, 폐기해서 샘플링할 데이터가 없는 경우 더 적극적으로 공간적 디더링을 사용하는 등의 방법으로
한 프레임의 샘플링이 인접 픽셀과 크게 다른 결과를 만드는 것을 방지합니다.